TILERA®

MULTICORE DEVELOPMENT ENVIRONMENT USER GUIDE

DOCUMENT RELEASE 1.30
DOC. NO. UG501
MARCH 2014
TILERA CORPORATION

Publishing Information:

Document Number	UG501
Doc. Release Number	1.30
Date	12 March 2014

Contact Information:

Tilera Corporation
Information info@tilera.com **Web Site http://www.tilera.com**

CONTENTS

PREFACE

About This Guide

This guide describes how to use Tilera's Multicore Development Environment™ (MDE). It offers instructions for creating, running, debugging, and profiling applications using the MDE command-line tools.

This guide is relative to the MDE Release 4.2.

Intended Audience

This guide is intended for use by Tile Processor™ application developers.

Note: This guide assumes that you have already become familiar with *Getting Started with the Multicore Development Environment* (UG504) and with *Programming the TILE-Gx Processor* (UG505).

Changes from the Previous Release

The major changes in this guide from MDE 3.0 are:

- The information about the Tilera Integrated Development Environment (IDE) has been moved to a separate manual. See *Using the Tilera IDE* (UG511).
- The former chapter called "What Is the Tilera Multicore Development Environment (MDE)?" has been moved to *Getting Started with the Multicore Development Environment* (UG504).
- Some information about cross-compilation, using tile-monitor to run programs, debugging, and profiling has been moved from *Programming the TILE-Gx Processor* (UG505) to this manual.
- Section 7.3.4 "Specifying Hardware Events to Record" on page 42 has been updated.

Document Organization

This guide is organized as follows:

- Chapter 1: Using Tools in the MDE Environment provides an introduction to tools and how to use them in the MDE environment.
- Chapter 2: Using tile-monitor provides an introduction to the `tile-monitor` tool to run applications at the command line. (Most of the information about `tile-monitor` is found in the `man` page.)
- Chapter 3: Running Applications on the Simulator describes how to use the simulator. (This chapter contains some information not found in the `man` page.)
- Chapter 4: Compiling and Running C and C++ Applications at the Command Line describes how to use the `tile-gcc` and `tile-g++` commands to run applications at the command line.
- Chapter 5: Debugging provides information about debugging Tilera applications.

- Chapter 6: Viewing Tile Processor Architecture Traces Using tile-trace-view describes how to generate traces and use the `tile-trace-view` graphical viewer to view them.
- Chapter 7: Collecting and Analyzing Performance Information describes how to collect performance information at the command line and analyze it.
- Chapter 8: Tools for Remote Access and Networking describes tools used for remote access and networking.

The guide also contains an Index.

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tilera tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own Tilera IDE Online Documentation, which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command Help → Help Contents.

- When the online help browser is displayed, select the document Tilera IDE Online Documentation in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at http://www.tilera.com.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[ ]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
...	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as root.
command	A command name appears in the Courier New font
computer output	Computer output appears in the Courier New font.
filename	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
new term	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tilera tools and examples. We recommend that you use the tile-env script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

CHAPTER 1 USING TOOLS IN THE MDE ENVIRONMENT

This chapter provides an introduction to tools and how to use them in the MDE environment:

See also Chapter 8: Tools for Remote Access and Networking.

1.1 Available Tools

This section describes some of the tools available in the MDE.

Some tools are available only on the x86 Linux side (the host side). Others are available on the x86 Linux side and the Tile side.

Each user who wants to use the tools on the x86 Linux side should add the following (note the use of backticks, not single quotes) to their `$HOME/.profile` or `$HOME/.cshrc` file:

```
eval `/opt/tilera/TileraMDE-version/arch/bin/tile-env`
```

You can also dynamically enter this command in any shell.

1.1.1 Tools Available On Both x86 Linux Side and Tile Side

These tools include:

- GCC environment

  The GCC environment includes the `tile-gcc` and `tile-g++` cross-compilers, `tile-as` and `tile-ld` assembler and linker, and a variety of other standard Linux tools (`tile-addr2line`, `tile-nm`, `tile-objdump`, and so forth). These tools provide everything that a command-line developer needs to build applications.

  See Chapter 4: Compiling and Running C and C++ Applications at the Command Line.

- `tile-monitor`

This tool provides an integrated command-line interface for running programs on the Tile Processor hardware or simulator. In addition to support for booting the chip, uploading files, and mapping the host filesystem into Tile Linux, `tile-monitor` includes advanced support for debugging and profiling multicore applications.

See Chapter 2: Using tile-monitor.

- `tile-gdb`

  This is the standard, open-source `gdb` debugger with support for the Tile Processor architecture. While this can be used directly, we generally recommend that users take advantage of the integrated debugging interfaces provided by `tile-eclipse` or `tile-monitor`.

  See Chapter 5: Debugging.

- OProfile

  This tool is a Tilera version of the standard, open-source OProfile profiling infrastructure. This includes `tile-opannotate` and `tile-opreport`. We generally recommend that users use the integrated profiling interfaces provided by `tile-eclipse` or `tile-monitor`.

  See Chapter 7: Collecting and Analyzing Performance Information.

- `tile-reboot`

  This tool lets you reboot a network-connected Tile board.

  See "Reboot and/or reset boot image on network-connected Tilera boards (tile-reboot)" in the MDE Document Index.

- `tile-mkboot`

  This tool lets you create a stream of executable code and data that can be used to boot a Tile Processor.

  See "Tile Processor Boot Image (tile-mkboot)" in the MDE Document Index or see the man page (these two files are identical).

- `tile-sbim` (on x86 Linux side) and `sbim` (on Tile side)

  This tool lets you manage the contents of a bootable SPI ROM (SROM).

  See "Tile Processor SROM Boot Image Manager (sbim, tile-sbim)" in the MDE Document Index.

- `binutils` (GNU Binary Utilities)

  See "Tile-Gx Processor Binary Utilities" in the MDE Document Index.

1.1.2 Tools Available Only on x86 Linux Side

These tools include:

- Eclipse IDE (`tile-eclipse`)

  Tilera's custom version of the open-source Eclipse IDE provides a GUI interface for all phases of program development, including authoring, building, running, debugging, and profiling. It includes Tilera-specific extensions for running programs on TILExpress boards, enhanced multi-process debugging, and detailed profile analysis.

  See *Using the Tilera IDE* (UG511).

- `tile-console`

  This kermit-based terminal can be used to access the Linux console that runs over the UART port of TILEncore-Gx boards.

  See "Tile Processor Console (tile-console)" in the MDE Document Index.

- `tile-sim` simulator

  This is a software simulator for the Tile Processor. The simulator provides cycle-accurate profiling and tracing, often used to debug operating system or driver code. In most cases, `tile-sim` is launched by means of `tile-monitor`, which provides a uniform interface for running programs on either hardware or the simulator.

  See Chapter 3: Running Applications on the Simulator.

- `tile-trace-view`

  This is Tilera Trace Viewer, a GUI viewer.

  See Chapter 6: Viewing Tile Processor Architecture Traces Using tile-trace-view.

1.2 Getting Help on the MDE Tools

The MDE tools generally follow the Linux convention of providing a `--help` option that summarizes how to use the command. For example:

```
$ tile-gcc --help
```

The `tile-monitor` tool has a similar command:

```
$ tile-monitor --help
```

which mentions these additional kinds of help:

```
$ tile-monitor --help-options
$ tile-monitor --help-commands
$ tile-monitor --help-examples
$ tile-monitor --help-stdin
$ tile-monitor --help-all
```

The MDE command-line tools also have man pages. For example:

```
$ tile-man tile-gcc
```

There is also a set of HTML files in the MDE Document Index that provides information about the tools. Some of these HTML files correspond to the man pages, and some provide more comprehensive information than the man pages, and some provide information that is not available in man-page format.

1.3 Tools in the $TILERA_ROOT/tile/ Directory

Getting Started with the Multicore Development Environment (UG504) describes the tarballs that are provided for the MDE installation. More than 2,000 RPMs are available with the installation. After installation, these RPMs are found in the `$TILERA_ROOT/tile/` directory of the MDE.

A minimal subset of Linux commands is provided in the boot environment of the shipped Linux image. By default, an initramfs boot (instead of the other option, a disk-based boot) is performed.

The file hierarchy in the `$TILERA_ROOT/tile/` directory of the MDE includes many more commands that are not present in the standard `initramfs` subset. For example, you can find the native C and C++ compiler, other language support such as Perl and Python, the `gdb` debugger, the `vi` editor, the `ext2` and `DOS` filesystem tools, and many others.

The easiest way to gain access to the files in the `$TILERA_ROOT/tile/` directory is to use `tile-monitor --root`. This command mounts and upload the interesting subdirectories of `$TILERA_ROOT/tile/`.

1.4 Searching the Tilera Documentation

The MDE 4.0 release provides a small, standalone documentation search server, which you can use to perform a keyword search on Tilera documentation, including our PDF files, TILE tools man pages, and example code files.

The documentation search server uses a pre-built index. Currently, the index covers all of the files in these directories:

- `$TILERA_ROOT/tile/usr/tilera/doc/pdf`
- `$TILERA_ROOT/tile/usr/tilera/doc/html`
- `$TILERA_ROOT/tile/usr/tilera/examples`

1.4.1 Starting the Documentation Search Server

To search the documentation, tool, you must first start the documentation search server, by running the `tile-doc-search` command.

The `tile-doc-search` command starts the server, and reports a URL by which you can access the search documentation search server from your favorite web browser:

```
$ tile-doc-search
Started Tilera doc server process, pid = 24831.
Doc server URL: http://server_name:8080
```

Note: The documentation search server is a small Jetty web-server which uses the open-source Lucene search API, running in a Java servlet container.

Setting the Documentation Search Server Port Number

The `tile-doc-search` server port number defaults to `8080`. To override the documentation search server port number, use the `--port` *`number`* option (abbreviated as `-p`) as in this example:

```
$ tile-doc-search -p 8081
Started Tilera doc server process, pid = 27382.
Doc server URL: http://server_name:8081
```

1.4.2 Stopping the Documentation Search Server

To stop the server instance, run the `tile-doc-search` command again.

The documentation search server stores the server PID of your server instance in a `/tmp` file. If you try to start the server and it reports an attempt to shut down a non-existent process, just repeat the command, since the message indicates that the server is cleaning up the PID file that was lying around.

1.4.3 Connecting to the Documentation Search Server

From a browser, connect to the indicated URL (`http://`*`server_name`*`:8080`).

The browser's home page provides a **Documentation Search** area, where you can enter keywords to search for. You can just enter a set of keywords and click the **Search** button. You can also use modifiers to specify a more exact search; these are described below.

The home page also includes links to documents in the MDE documentation set; click on these to display the documents in the browser.

1.4.4 Searching for Terms in Tilera Documentation

You can search for any number of space-separated terms. The search tool returns matches for any of the terms. The search tool ranks matches having multiple terms in close proximity as higher in relevance. We define these categories of documentation to search:

- manuals, primarily Adobe Acrobat format
- man pages, which are both man pages and HTML files
- examples, as are available in example code, header and support files

If a search phrase includes spaces, enclose it in double quotes to search for the entire phrase as a unit.

You can require the documentation search tool to find a particular term or phrase, or to exclude that term or phrase. Specify '+' character prefix to require a search item be present, and the '-' character to exclude it. Table 1-1 shows how to use the basic documentation search options.

Table 1-1. Using Standard Search Methods in the Documentation Search Tool

Searching for	Results
`profile view`	finds "profile" or "view"
`Profile View`	finds "profile" or "view" (searches are case-insensitive)
`"Profile View"`	finds "Profile View"
`+"profile view"`	requires that "Profile View" be present
`profile -view`	finds "profile" but excludes "view"
`\--resume`	finds "--resume" option
`"--resume"`	finds "--resume", the same as escaping the dashes

Using Wildcards, Special Characters and Boolean Operators

Use the '?' character as a wildcard for a single character, and '*' as a wildcard for any number of characters.

Note: You cannot use a wildcard character at the beginning of a term. Also, you can only use wildcards in single-term searches, not multi-word phrases.

When searching for terms with underscores in the names, such as function names, enter the entire name, or use wildcards.

Table 1-2 shows examples of using wild cards and special characters.

Table 1-2. Using Wildcards and Special Characters in the Search Tool

This search string	Results
`ch?p`	finds "chip"
`tile??`	"tile" and "Tilera"

Table 1-2. Using Wildcards and Special Characters in the Search Tool

This search string	Results
`TILE*`	finds "TILE", "TILEmpower" and "TILEncore"
`int func\(int arg\)`	an example of escaping with backslash
`"int load(char* argc)";`	an example of escaping with quotes
`tmc_cpus_get_my_cpu`	finds tmc_cpus_get_my_cpu
`tmc_cpus_get*`	finds tmc_cpus_get_my_cpu, tmc_cpus_get_my_affinity, etc.
`GNU_SO*`	finds GNU_SOURCE
`*ILE`	not supported
`multi w*rd`	not supported

To include special characters (+, -, *, ?, (,), etc.) in your search, escape them with back-slashes, or enclose them in quotes.

You can use `AND`, `OR`, and `NOT` (or equivalently "`&&`" for AND, and "`||`" for OR) to form boolean searches.

Table 1-3 shows examples of using boolean operators in the documentation search tool.

Table 1-3. Using Boolean Operators in the in the Documentation Search Tool

This search string	Demonstrates
`profile AND view`	finding "Profile View"
`profile AND \( NOT view \)`	finding only "profile"

Biasing a Search

You can bias the search towards a specific type of documentation, or to exclude that type.

Table 1-4 shows the arguments you can use in the search box to focus the search activity.

Table 1-4. Focusing a Search for Document Type or Extension Type

This search argument	Results in the Documentation Search Tool
`type:manual`	favoring matches in PDF manuals
`type:manpage`	favoring matches in man pages and HTML files
`type:example`	favoring matches in example code, header and support files
`type:other`	favoring matches in any other document types
`+type:example`	restricting search to only match example files

Table 1-4. Focusing a Search for Document Type or Extension Type

This search argument	Results in the Documentation Search Tool
`-type:example`	restricting search to exclude example files
`ext:c`	favoring matches in `.c` example files
`+ext:c`	restricting the search to only match `.c` example files

1.5 Run-Time Libraries

Tilera provides several run-time libraries with Tilera-specific application programming interfaces (APIs) that you can use to write programs that run on the Tile Processor.

These libraries (among others) are documented in the *Application Libraries Reference Manual* (UG527):

- Tilera Multicore Components (TMC), which can be used to build parallel programs. The TMC is composed of a number of APIs that include routines for managing the binding of tasks to cores, allocating memory with particular performance characteristics, and communicating between tasks.
- GXIO low-level I/O device control API, which provides applications direct access to the TILE-Gx I/O shims.
- GXCR user-level Crypto Acceleration API, which provides a software programming interface to hardware acceleration of ciphers and hash functions.
- GXPCI PCIe and StreamIO data transfer API, which provides application control of data transfer operations among the PCIe ports using the PCIe and StreamIO protocols.

Tilera also provides standard libraries required by the compiler and linker environment, including the default C library (`glibc`, which contains the libraries `libc`, `libm`, `libdl`), and the standard C++ libraries (`libstdc++`).

1.6 Tilera Tools for System Monitoring

This section describes a tool you can use for system diagnostics.

1.6.1 Using mcstat to Monitor Memory Controller Usage

`mcstat` (Memory Controller Statistics) provides a convenient way to monitor memory activity.

Tile Processor™ memory is accessed by means of up to four memory controllers. `mcstat` reports statistics for each memory controller over some period. The duration of this period is controlled by command-line options:

- `-c` — Duration is the time it takes to run the given command.
- `-i` — Duration is the given number of seconds. `mcstat` repeatedly monitors intervals of the given number of seconds.
- `-w` — Duration is the given number of seconds. `mcstat` repeatedly monitors intervals of the given number of seconds. (Similar to `-i`, but `-w` displays more output.)
- `-t` — Duration is the given number of seconds. `mcstat` measures one such interval.

For more information, see "Using mcstat for Memory Controller Statistics" in the *Multicore Development Environment Optimization Guide* (UG515).

1.7 Packages Provided with an Initramfs Boot

The most important package provided with the `initramfs` boot is the `busybox` tool set. This is a single binary (provided in `/bin/busybox`) that provides the functionality of many standard Linux tools all in one. The various utilities are provided as symbolic links (symlinks) pointing at `/bin/busybox`. For example, `/bin/ls` is simply a symbolic link pointing at `busybox`.

The shell provided with `busybox` and used in Tilera Linux is the `ash` shell. The `ash` shell is more powerful than the traditional `sh` shell but more compact than the full-featured GNU `bash` shell. For details about the `ash` shell, you can run `man ash` on the host Linux system where the MDE is installed.

Other important packages provided with an initramfs boot are:

- `dropbear 0.52`

  This package provides `ssh` server and client support in the default shipped MDE. Note that the same private key is used for all shipped servers, which allows you to login to the `root` account with no password.

- `ps`

  Lists the currently running processes or threads.

- `shepherd`

  For information, see Section 2.2 “Shepherd Process” on page 12.

- `tile-sbim`

  Used to manage the contents of a bootable SROM.

- `taskset`

  Used to set or retrieve the CPU affinity of a running process given its PID or to launch a new COMMAND with a given CPU affinity.

For a complete list of files provided with an `initramfs` boot, see the file `$TILERA_ROOT/tile/usr/lib/initramfs/default/contents.txt`.

1.8 Using Tilera Special /proc Files

Many special files are available in the `/proc/tile` and `/proc/sys/tile` hierarchies. Examining these files can provide insight into the running Linux kernel. The set of files is documented in the “Linux Interfaces and Customization” chapter in the *Multicore Development Environment System Programmer’s Guide* (UG509).

Some of the files that are relevant for command-line system monitoring are:

- `memprof` — The `/proc/tile/memprof` file provides an interface for measuring the throughput of a Tile Processor’s memory controllers. It includes support for starting, stopping, and clearing the memory profile, as well as reading the results. For more information, see “Memory Bandwidth Profiler” in the Tilera MDE Document Index.
- `environment` — The `/proc/tile/environment` file reports the chip and board temperature. This file’s main user is the `tempmond` daemon, which will shut down the board automatically if the temperature goes too high.
- `memory` — The `/proc/tile/memory` file displays the speed of each memory controller.

- `interrupts` — The `/proc/tile/interrupts` file can be used to track what type of interrupts are being delivered to each CPU. This can help establish why applications with low latency requirements experience higher latencies than desired. The different categories of interrupts are:
  - `timer` — Timer interrupts, typically for the Linux task scheduler, or for delivering time events to user space
  - `syscall` — Number of system calls issued on this CPU
  - `resched` — Number of inter-processor interrupts (IPIs) delivered to cause this CPU to run the scheduler
  - `hvflush` — Number of times another CPU requested the hypervisor to perform a TLB or cache flush on this CPU
  - `SMPcall` — Number of generic IPIs delivered to this CPU to cause remote function calls to run
  - `hvmsg` — Hypervisor device messages delivered to this CPU
  - `devintr` — Hypervisor device interrupts delivered to this CPU

1.9 Using the System Console

The system console is a special text interface to the Tile Processor that displays diagnostic and crash information and allows for shell access.

1.9.1 Displaying Kernel Messages

The console displays hypervisor output, primarily during early bootup. The hypervisor always displays a startup banner as it boots up. Additional debugging information can be shown by setting the `debug` option in the `hvconfig` file.

Linux also displays copious bootup information on the console. In addition, you might occasionally see status messages on the console (for example, messages relating to Linux home caching). In the unlikely event of a Linux kernel panic, all the diagnostic information is displayed on the console prior to the kernel halting. This output can be extremely valuable for problem diagnosis.

Some user applications also send some output to the console when running. For example, the `udhcpc` client (used to acquire a network address from a DHCP server) displays some information to the console while it is starting up. Additionally, the `tempmond` process displays a warning message on the console when the Tile Processor or board reaches a dangerously high temperature.

If you add `quiet TLR_QUIET=y` to the boot arguments, Linux displays less bootup information. The former filters out more of the kernel's own messages; the latter arranges for some normal Linux applications (such as `udhcpc`) to produce less output when run.

The `/proc/sys/tile/crashinfo` file controls whether or not Linux displays application-level crash information on the console. If an ASCII 1 (one) is written to this file, Linux displays signal and register information, a stack trace, and a memory dump if relevant for each application at the same point where it would normally create a core dump (if enabled).

The Linux kernel console output can be monitored (for example, from a `telnet` session) by running `cat /proc/kmsg`. In this case, the severity level of each kernel message is shown. However, note that a panic message will likely not make it to `/proc/kmsg` prior to the system shutting down. Also, hypervisor and user output to the console is not shown in `/proc/kmsg`.

Tilera recommends that the console always be displayed.

1.9.2 Using the Interactive Console by Default

By default, the Linux boot enables access to a shell prompt on the console. The user presses Enter on the console to launch the console shell. (If you want no shell on the console, specify the `TLR_INTERACTIVE=n` boot environment variable.)

The launched shell runs in a full `tty` environment, so job control, keyboard signals, and the like, are fully functional.

If you exit from the shell (for example, by running exit or entering Control-D), the Linux init process loops back and displays the `Please press Enter to activate this console` message again.

1.9.3 Configuring Additional Users or Passwords

By default, Tilera Linux ships in an "open" security configuration, where `root` has no password, and a `root` shell is provided on request on the serial console.

Additional user accounts can be added by editing `/etc/passwd` and `/etc/shadow` to add additional lines. Passwords can be added by running the `passwd` command or by copying the encrypted password field in `/etc/shadow` from another Linux system.

To change the console to run a login prompt instead of a shell, the tile `/etc/inittab` file can be modified to require a login by running `/bin/login` instead of `-/bin/sh` on the console; see the comments in that file.

1.9.4 Magic SysRq Linux Key: Control-O

By default, Tilera Linux is configured to support the Magic SysRq key sequence. This is the Control-O key. You can press Control-O followed by various other characters to generate kernel-level debug and status information, sync the filesystem, shut down the system, and so forth. Press Control-O then `h` to display the help information.

1.9.5 Miscellaneous Diagnostics

A console connection is also required for the low-level chip diagnostic tools, such as `tile-btk`. In this mode the serial console converts to running a "protocol mode" to send encoded packets to the Tilera hardware shim that normally drives the serial device.

CHAPTER 2 USING TILE-MONITOR

This chapter describes `tile-monitor`:

Note: `tile-monitor` is well documented in the sources described in Section 2.3 "For More Information" on page 13. This chapter is short because it does not duplicate that information.

2.1 What Is tile-monitor?

`tile-monitor` is a tool to support command-line interaction with a Tile Processor™ from an x86 host. It allows you to interact with a Tile Processor without actually logging into that Tile Processor. It provides a single, integrated interface for many of the common tasks that are built into the MDE toolchain, including:

- Building a boot image and booting a TILE-Gx™ processor, TILEmpower-Gx™ developer platform, TILEncore-Gx™ Intelligent Application Adapter, or the MDE simulator
- Transferring files between the development host and the Tile environment
- Specifying the tiles on which an application should run
- Mounting host files into the Tile Linux filesystem
- Running applications
- Launching debuggers when an applications crashes
- Running the OProfile profiler

The syntax is:

```
$ tile-monitor [options ...] [commands...] [-- exe [args ...]]
```

For example, you use the `tile-monitor` command to attach to hardware, such as the TILEmpower-Gx, on a network IP address as follows:

```
$ tile-monitor --net <host>:<port>
```

For another example, the following command will boot a Tile board, upload a compiled `hello` program, and run it:

```
$ tile-monitor --dev [usbN|gxpciN] --upload hello hello -- hello
```

`tile-monitor` is available when running on a host system and also when running natively on a TILE-Gx Processor. However, when running on a TILE-Gx Processor, the set of available options is different.

Many of the interesting things done by `tile-monitor` are actually done by other processes.

Booting requires a bootrom file, which you create using the `tile-mkboot` command. The `tile-reboot` command reboots a networked Tile Processor. The `tile-sim` command simulates a Tile Processor environment. On the Tile Processor, a special shepherd process does most of the actual work involved in file transfers and running and debugging applications, and the `opro-file` daemon performs most of the actual work involved in hardware profiling.

`tile-monitor` processes all of its command-line options, then handles any requested booting operations and connects to the shepherd process running on the Tile Processor, then processes any command-line commands, and finally enters an interactive mode and waits for you to enter more commands.

You can use the `tile-monitor` command to modify the boot image that starts the Tile Processor. Use the `--hvx` parameter to specify Linux boot parameters. For example, the following command adds `dataplane=1-35` to the Linux parameters for a Tile-Gx8036™ processor, causing Linux to run ZOL dataplane tiles on every core except core zero:

```
$ tile-monitor --hvx dataplane=1-35
```

The `tile-monitor` command also supports complete replacement of the hypervisor configuration file via `--hvc`, booting a custom Linux image by means of `--vmlinux`, and a custom hypervisor by means of `--hv-bin-dir`. For more information, see the *Multicore Development Environment System Programmer's Guide* (UG509).

Many other `tile-monitor` command-line options are available. See the `tile-monitor` manpage (enter `tile-man tile-monitor` at the command prompt). An HTML version of the manpage is available in the MDE Document Index in the "Target Platform Tools" section.

2.2 Shepherd Process

A shepherd is a special process that can watch other processes, tracking both the forking of child processes and the creation of threads, and treating the resulting set of processes as a single application.

The shepherd runs on the Tile Processor. It supports application development by communicating with the host-side `tile-monitor` and the IDE. (This is known as a "monitored" shepherd, and is the typical way the shepherd runs.)

The shepherd controls and reads the status of processes running on the tiles and provides this information back to the `tile-monitor` process running on the host, helping `tile-monitor` manage (and debug) processes and their consoles. For example, the default `/etc/rc` script (for an `initramfs` boot) or `/etc/init/tilera.conf` (for an Upstart boot) normally starts a shepherd, which can handle PCI, TMFIFO, or network connections, as appropriate.

You can also use the shepherd to provide services to applications running on the Tile Processor. For example, the shepherd supports the use of the FUSE filesystem and can arrange for parts of the host's file systems to be mounted within the Linux directory hierarchy.

The shepherd can be started by running `/bin/shepherd` with arguments telling it how to communicate with a `tile-monitor` process on the host.

Note: A “local” shepherd (which does not use `tile-monitor`) can be started by running `/bin/shepherd`, with optional `--tile` arguments, followed by an initial executable and its arguments. This local shepherd exits once the application has exited, with an exit status based on how the processes in the application exited. If any process crashes (or exits with an exit code between 128 and 255), the local shepherd kills all of the other processes in the same application.

2.3 For More Information

See these sources for information about `tile-monitor`:

- The `tile-monitor` help:

```
$ tile-monitor --help
$ tile-monitor --help-options
$ tile-monitor --help-commands
$ tile-monitor --help-examples
$ tile-monitor --help-stdin
$ tile-monitor --help-all
```

- ”Tile Processor Monitor” (`tile-monitor.html`) in the “Target Platform Tools” section of the MDE Document Index or the man page at `tile-man tile-monitor`. (These are identical.)

CHAPTER 3 RUNNING APPLICATIONS ON THE SIMULATOR

This chapter describes how to run applications on the simulator:

3.1 Overview

For basic information about running the simulator, see the "Tile Processor Simulator" (`tile-sim.html`) in the "Development Tools" section of the MDE Document Index or the man page at `tile-man tile-sim`. (These are identical.)

Note: Do not run `tile-sim` at the command prompt. However, see the `tile-sim` documentation for information about simulator arguments for `tile-monitor`.

For information about how to generate a simulator profile data file, see the section on "Generating a Profile Data File Outside the IDE" in *Using the Tilera IDE* (UG511).

3.2 Syntax

Use `tile-monitor` to run an application on the simulator. For example:

```
$ tile-monitor --simulator --image 2x2 \
   --upload compute /tmp/compute \
   --run -+- /tmp/compute /tmp/results -+- \
   --download /tmp/results results \
   --quit
```

In this example, the simulator does the following:

1. Loads a prebuilt image of a 2x2 Tile Processor. See Section 3.3 "Choosing an Image File for the Simulator" on page 16.
2. Uploads the executable `compute` from the development host to the Tile Processor.
3. Runs the executable and writes the results to the `results` file.
4. Downloads the `results` from the Tile Processor and saves it in the current directory.
5. Exits.

There are many `tile-monitor` options that work only with the simulator (that is, they do not work on the hardware). The most common one is `--image`. See Section 3.3 "Choosing an Image File for the Simulator" on page 16.

Another important simulator option is `--functional`, which enables functional mode. The simulator defaults to timing-accurate mode, but also supports a faster mode called functional mode. Functional mode reduces simulation time at the cost of not generating cycle-accurate performance information.

Other options require that you use the `--sim-arg` or `--sim-args` option to separate them from other `tile-monitor` options:

```
$ tile-monitor --sim-arg simulator-option
$ tile-monitor --sim-args -+- { simulator-option ... } -+-
```

Note: When you use the `--sim-args` option, delimit the simulator options by the dash/plus sign/dash (`-+-`) combination.

When used on the command line, all commands that take a variable number of arguments must wrap their arguments in a pair of matching delimiters. The delimiters can be any string starting with a dash that is unlikely to match any actual argument, including a single dash. In interactive mode, these delimiters are not needed and are not allowed.

On the command line, you can include additional `tile-monitor` commands after the delimiters.

For example:

```
$ tile-monitor [...] --sim-arg --trace-lines --sim-arg --trace-disasm [...]
```

or:

```
$ tile-monitor [...] --sim-args -+- --trace-lines --trace-disasm -+- [...]
```

You can also specify simulator options by getting a `tile-monitor` command prompt and then entering commands like this:

```
Command: sim sim-option
```

3.3 Choosing an Image File for the Simulator

An *image file* is a file containing a binary snapshot of the state of the simulator at a particular point, typically after the operating system has finished booting and before the application has started running. The binary snapshot contains a particular *chip configuration*, or arrangement of tiles. You use an image file to save time, especially with larger chip configurations, since booting a simulation of the full chip takes time.

When you use an image file to boot the simulator, you cannot use some simulator command-line-only options, because those options apply only if you are booting a simulation of the chip from scratch.

Tilera supplies pre-built image files for common chip configurations, including the following:

- `1x1.image`, `2x2.image`, `4x4.image` — simulator images that configure the corresponding tile grids, and no I/O devices.
- `gx8036.image` — a simulator image that configures a tile grid and the memory controllers corresponding to an actual TILE-Gx8036™ processor.
- `gx8036-dataplane.image` — a simulator image that configures tile 0,0 for non-dataplane operations and all other tiles set aside for dataplane mode running ZOL.
- `gx8016.image` — a simulator image that configures a tile grid and the memory controllers corresponding to a TILE-Gx8016™ processor.

The `gx8016.image`, `gx8036.image`, and `gx8036-dataplane.image` files define the hardware resources of the fully-functioned Gx-8*xxx* family of parts. You can also use them to simulate the Gx-5*xxx* families, with the appropriate number of tiles.

The Tilera-supplied image files reside in the boot directory:

```
$TILERA_ROOT/lib/boot/
```

Use the `--image` or `--image-file` option to specify the chip configuration you want.

To specify an image by path, use:

`--image-file` *`path`*

To specify an image by name, use:

`--image` *`name`*

where `--image` *`name`* is shorthand for `--image-file $TILERA_ROOT/lib/boot/`*`name`*`.image`.

For example, to use the image file that specifies the 2x2 configuration, use:

```
$ tile-monitor --simulator --image 2x2
```

3.4 Simulator Options Used for Debugging

Table 6 shows some simulator options that provide useful debugging information about the simulated chip and the processes running on it.

Table 6. Simulator Options Useful in Debugging

Commands	Examples	Description
`after CYCLES do COMMAND`	`after 1000 do trace none`	Enqueues a command to be executed once the simulator reaches cycle number `CYCLE`.
`bt`		Displays the current stack trace of all simulated processes.
`debug`	`on 0,0 debug`	Requests a tile-level `tile-gdb` session be created for each tile in the focus. Note that this is a lower-level debugging technique intended for debugging tile error conditions rather than general user-level applications.
`dump FLAG...`	`dump dtlb`	Dumps a description of machine state on tiles in the current focus. Flag names might be abbreviated when unambiguous. Flags include: `all` All tracing flags. `backtrace` Backtrace of current process. `dtlb` Data TLB. `itlb` Instruction TLB. `l1d` L1 Data Cache. `l1i` L1 Instruction Cache. `l2` L2 Cache. `regs` Registers. `sprs` SPR values.
`exit STATUS`	`exit 1`	Terminates the simulator process with given exit status. See also the `quit` command, which is equivalent to `exit 0`.

Table 6. Simulator Options Useful in Debugging (continued)

Commands	Examples	Description
`focus TILE...`	`focus 3,2 4,7`	Specify tiles to which future commands apply. Each TILE is an x,y pair like 3,2. Note that the focus is only relevant to some commands. To apply one command to a set of tiles without modifying the default focus, see the `on` command.
`freeze`		Stops simulation of the entire chip until `unfreeze`.
`help [COMMAND_NAME]`	`help focus`	With no argument, this provides a brief listing of all possible commands. With one argument, it provides detailed help on that command. `help all` provides detailed help on all commands.
`on TILE... do COMMAND`	`on 0,0 do dump regs`	Temporarily set the focus to `TILE...`, as with the `focus` command, and then execute COMMAND. Once COMMAND finishes the original focus is restored.
`panic`	`on 0,0 panic`	Bare-metal debug if the simulated OS panics. If `bm-debug-on-panic` has been set to true using either `set bm-debug-on-panic true` or `tile-monitor --bm-debug-on-panic`, then this initiates a `tile-gdb` session for each tile in the focus. Otherwise it does nothing.
`profiler chip_SUBCOMMAND FLAG...`	`profiler chip_enable all`	Specify a chip-level profiler subcommand. The subcommand affects all tiles, not only tiles in the current focus. You can abbreviate subcommand and flag names when they are unambiguous. Possible subcommands are: • `chip_clear` • `chip_disable` • `chip_enable` The possible flags are: • `all`: All chip-level profiling flags. • `memctl`: Memory controller profiling flag. • `xaui`: XAUI controller profiling flag. • `pcie`:PCIe controller profiling flag.
`profiler SUBCOMMAND`	`profiler enable`	Specify a profiler subcommand. The subcommand affects only tiles in the current focus. You can abbreviate subcommand and flag names when they are unambiguous. The possible values for SUBCOMMAND are: • `clear` • `disable` • `enable`.
`quit`		Terminates the simulator process with exit status 0.

Table 6. Simulator Options Useful in Debugging (continued)

Commands	Examples	Description
`set VAR VALUE`	`set functional true`	Sets the value of a special tile-sim state variable to the specified value. False boolean values are specified with either 0 or false, and true boolean values are specified with either 1 or true. Possible VAR names are: • `bm-debug-on-panic Boolean`: Debug if the simulated OS panics. • `functional Boolean:` Specify functional simulation mode. • `verbosity Integer:` Adjust the verbosity of simulator messages.
`trace FLAG...`	`trace l2_cache`	Specify which events should generate simulator trace output. This currently affects all tiles, not just those in the focus. Flag names might be abbreviated when unambiguous. Possible flags are: • `all`: All tracing flags. • `cycles`: Prefix trace lines with current cycle number. • `disasm`: Emit disassembly of executed code. • `l2_cache`: Trace L2 cache transactions. • `lines`: Emit source line info of executed code. • `memory_controller`: Trace memory controller transactions. • `none`: No tracing. • `register_writes:` Trace changes to register values. • `router`: Trace router information on mesh networks. • `stall_info:` Display processor stalls.
`unfreeze`		Resumes simulation on all tiles.

3.4.1 Two Debugging Options for the Simulator

These two options are used for debugging the simulator:

- `--grind-coherence`

  Enables additional checks to help find shared memory cache coherence bugs in the simulated program(s). (This option slows down simulation considerably.)

- `--grind-warnings-not-fatal`

  Do not exit immediately on the first `grind-coherence` warning (`--grind-warnings-fatal` is on by default and this turns it off).

3.5 For More Information

See "Tile Processor Simulator" (`tile-sim.html`) in the "Development Tools" section of the MDE Document Index or the man page at `tile-man tile-sim`. (These are identical.)

CHAPTER 4 COMPILING AND RUNNING C AND C++ APPLICATIONS AT THE COMMAND LINE

This chapter describes how to use the `tile-gcc` and `tile-g++` commands to compile and run applications:

Note: For information on using the optimization features of the `tile-gcc/g++` compiler, see the *Multicore Development Environment Optimization Guide* (UG515).

4.1 Overview of tile-gcc and tile-g++

Cross compilation refers to building on one platform (called the build platform), and running the resulting program on another platform (usually called the target).

`tile-gcc` and `tile-g++` are the cross-platform ports of the GNU C and C++ compilers for the Tile Processor™. These compilers run on x86 machines and generate Tile Processor code.

You can compile an application using the `tile-gcc` or `tile-g++` commands from the command line.

Note: For compatibility with previous releases, you can also use `tile-cc` or `tile-c++`, which are synonymous with `tile-gcc` and `tile-g++`, respectively.

`tile-gcc` compiles ANSI standard C to optimized machine code for the Tile Processor architecture. It handles all of ANSI C, including software emulation of operations on data types such as floating-point and 64-bit integers. A standard C run-time library is included.

`tile-g++` is the corresponding ANSI C++ compiler. It uses essentially the same options as the C compiler. The major difference is that C++ run-time libraries are automatically included in the link step with `tile-g++`. (Thus, the `-mnewlib` option is not supported on `tile-g++` for linking.)

Note: `-mnewlib` selects an alternate, simpler version of our C libraries. It has fewer features (for example no threads, shared libraries or C++ support), but comes with a different license some people find more attractive. The default C run-time library is `glibc`.

Because it is incompatible with C++, the flag does not make sense for C++ programs.

`tile-gcc` and `tile-g++` also act as a wrapper for other phases of the program compilation and build process, including the assembler (`tile-as`) and linker (`tile-ld`). Most users will not need to use these lower-level components directly.

Intrinsics are provided for all architecture-specific instructions such as SIMD instructions and special-purpose registers (SPRs). See Section 4.8 "Intrinsic Functions for All Instructions" on page 26.

4.1.1 Documentation for GCC

`tile-gcc` and `tile-g++` are based on GCC Version 4.4.7. For more information, see:

- The GCC user documentation for 4.4.7 here:

  `http://gcc.gnu.org/onlinedocs/gcc-4.4.7/gcc/index.html`

  or here:

  `$TILERA_ROOT/doc/html/gcc/index.html`

- The `tile-gcc/tile-g++` manpage. (Or you can click on "Tile Processor C and C++ Compiler" under "Development Tools" in the MDE Document Index, which is identical to the manpage.)

4.1.2 Syntax

The syntax for both `tile-gcc` and `tile-g++` is:

```
{ tile-gcc | tile-g++ } [ -c | -S | -E ] [-gdebug-option] [-Ooptimization-option]
[-Iinclude-directory...] [-Llibrary-directory...] [-Dmacro[=value]...] [-Umacro...] [-std=language] [-flang-option...] [-Wwarn-option...] [option...] [-o outfile] infile...
```

The command-line options for `tile-gcc` and `tile-g++` are largely compatible with those for `gcc` and `g++`. For a complete list of all arguments and values, see the GCC user documentation, described in Section 4.1.1 "Documentation for GCC" on page 22.

4.2 Source File Types

Suffixes of source file names indicate the language and kind of processing to be done, as shown in Table 1:

Table 1. File Types and Their Suffixes

Suffix	File Type	Standard Processing
`.c`	C source file.	Preprocessed, compiled, and assembled into an object file.
`.cc`, `.cpp`, `.cxx`, `.c++`, `.C`, `.CC`, `.CPP`, `.CXX`	C++ source file.	Preprocessed, compiled, and assembled into an object file.

Table 1. File Types and Their Suffixes

Suffix	File Type	Standard Processing
`.s`	Assembly source file.	Assembled into an object file.
`.S`	Assembly source file that must be preprocessed.	Preprocessed and assembled into an object file.
`.h`	Preprocessor header/include file.	Included when referenced by an `#include` line in a source file.
`.o`	Object file.	When linking, incorporated into executable.
`.a`	Library (archive) file.	When linking, contents incorporated into executable as needed.
`.so`	Shared library (shared object) file.	When linking, references incorporated into executable as needed.

4.3 Basic Operations: Compiling, Linking, and Running

Use the `tile-gcc` and `tile-g++` commands to build a program in one step, with a single command line of the following form:

```
$ tile-gcc -o executable [option ...] file.c ...
```

```
$ tile-g++ -o executable [option ...] file.cc ...
```

A typical flow is to break up the compilation and linkage steps, compiling one source file at a time to an object file, and then linking all the object files into an executable.

To compile, producing an object file:

```
$ tile-gcc -c -o file.o [option ...] file.c ...
```

To run the compiler and view the resulting assembly code without producing an executable, use the `-S` flag, which produces a `.s` file:

```
$ tile-gcc -S [option ...] file.c ...
```

To run just the C preprocessor and view the result of processing in-line directives, macros, and conditional compilation directives, use the `-E` flag, which prints the processed source file to the standard output:

```
$ tile-gcc -E [option ...] file.c ...
```

To link the object files into an executable:

```
$ tile-gcc -o executable [option ...] file.o ...
```

Note: If any C++ source files are involved, use `tile-g++` for the link step to include the C++ runtime libraries. Using the `tile-gcc` command to link object files that require C++ support results in link errors.

By default, `tile-gcc` and `tile-g++` both link with dynamic libraries.

If you do not want to use dynamic libraries, use the `-static` option when linking.

To build a shared library, use the `-fpic` option (not `-fPIC`) when compiling the object files (`-fpic` generates faster code).

4.4 Linking Libraries

To use features from the Tilera Multicore Components library (TMC™) and other provided APIs, the appropriate library files must be linked into the application, using the compiler's `-l` options. These library files are typically found in the `$TILERA_ROOT/tile/lib` directory, which is automatically searched by the compiler.

If the `-l`*`library`* option is specified, then the library file named `lib`*`library`*`.a` is linked into the application. (This is assuming that you use `-static` or that `libLIBRARY.so` is not available.) For example, `-lm` links in `libm.a`, the mathematics library.

To link against TMC, use `-ltmc`. To directly use this library, you can include various header files such as `tmc/cmem.h`, one for each component.

In addition, an application that makes uses of deprecated features such as `malloc_shared()` needs to link with `-ltmc`.

The standard C run-time library (or the standard C++ run-time library) is linked in automatically, and need not be specified explicitly.

4.5 Porting Applications

Most compilation commands are as simple as replacing `gcc` with `tile-gcc`. For example, to produce a Tile Processor™ binary that runs code specified in `hello.c`:

```
$ tile-gcc -o hello hello.c
```

Many applications use the `make` command to build their binary according to rules specified in `makefiles`. In many cases, rebuilding an application to run on the Tile Processor is as simple as overriding `make`'s `$(CC)` variable to point at `tile-gcc`. Consider the following trivial example makefile:

```
# Sample hello Makefile

hello: hello.c
        $(CC) -o $@ hello.c

clean:
        rm -f hello
```

In this case, the application can be ported to the Tile environment simply by passing `CC=tile-gcc` on the command line or by adding `CC=tile-gcc` to the makefile itself.

The MDE also provides a GNU `binutils` port for the Tile Processor, including tools like `tile-as`, `tile-addr2line`, `tile-objdump`, and the like.

For more information about cross-compilation, including suggestions for porting open-source applications that use `configure` scripts generated by Autoconf, see the "Compiling for the TILE Architecture" in the "Technical Notes" section of the MDE Document Index.

Note: If you are compiling a package that uses the GNU auto-configuration mechanism, it might not recognize the TILE target platform. If that is the case, search through the source tree for occurrences of the files `config.sub` and `config.guess`. Replace these files with the versions to be found in `$TILERA_ROOT/etc/`.

4.6 Performing Native Builds

Note: This section assumes that you are familiar with running the `tile-monitor` command from the command line. See Chapter 2: Using tile-monitor.

Conventional native compilation refers to building an application on the same platform on which it will run.

Cross-compiling software can be an extremely difficult task. This is especially true for certain open source software packages that use tools such as "GNU configure" to determine the build environment.

In these cases, it can be significantly easier to build the software natively, running the compiler tools on the Tile Processor rather than cross-compiling on the host system.

The tools available for native use are found in `$TILERA_ROOT/tile/` (for example, `$TILERA_ROOT/tile/usr/bin/gcc`).

The tools include the compiler (`gcc` and `g++`), related binutils tools such as and `ld`, BusyBox tools such as `cut` and `find` and `grep`, and a few special packages like `make`.

For example, to upload the `$TILERA_ROOT/tile/usr` directory from the host system to the Tile and then exit `tile-monitor`:

```
$ tile-monitor --upload-tile /usr --quit
```

By default, the `tile-monitor` command reboots the Tile target every time you run the command. If you don't want to reboot the Tile target, use the `tile-monitor --resume` option. With subsequent `tile-monitor` invocations, if you don't use the `--resume` option, your uploaded files will be lost.

After uploading the files, you can compile and run your program natively on the Tile target by using the syntax:

```
$ tile-monitor --resume --here -- cc -o myfile myfile.c
$ tile-monitor --resume --here -- myfile
```

Note: The `tile-monitor --here` option is a shorthand for `--mount` *`cwd rwd`* `--cd` *`rwd`*, where *`cwd`* is the current working directory on the host and *`rwd`* is the remote working directory on the target.

Other useful examples are:

```
$ tile-monitor --resume --here -- configure
$ tile-monitor --resume --here -- make
```

Note: The `tile-monitor -- exe [`*`arg ...`*`]` syntax is a shorthand for `--run -+- exe [`*`arg ...`*`] -+- --wait --quit`.

You can also perform more interesting tasks. For example, you can build `perl` by using commands such as the following:

```
$ tar -zxf .../perl-5.6.2.tar.gz
$ cd perl-5.6.2
$ tile-monitor --upload-tile /usr --here \
    --run -+- Configure -de -+- -- make
```

Alternately, you can simply mount the contents of the `tile/usr` directory for your installed MDE on the chip's `/usr` directory, rather than uploading all of it. To do this, run `tile-monitor` with the `--root` option, and leave that `tile-monitor` process running while you use the chip. While this is easier to do and faster to run initially, it is slower to run executables that are mounted this way, so it might be slower overall when you build using this option.

4.7 Using Unaligned Memory Reference Instructions

For information on unaligned memory reference instructions, see the chapter on "Unaligned Memory Accesses" in *Programming the TILE-Gx Processor* (UG505).

4.8 Intrinsic Functions for All Instructions

The compiler recognizes intrinsic functions corresponding to all the Tile Processor instructions. This permits close control over user code as well as access to the functionality of all the available instructions.

In general, these instructions are of the form:

```
y = __insn_clz(x);
```

which indicates that the variable *y* will be assigned the result of applying the `clz` instruction to the value in variable *x*.

There is a special case for post-increment instructions, of the form:

```
y = __insn_ld_add(p, 8);
```

In this case, the value of the pointer variable *p* is incremented by 8 bytes after the load from the address contained in *p* occurs.

For more information, see the section on "Using Intrinsics" in the chapter "Using the Optimization Features of the tile-gcc/g++ Compiler" in the *Multicore Development Environment Optimization Guide* (UG515).

4.9 Feedback-Based Optimization

Tilera provides feedback-based optimization for use with `tile-gcc/g++`. This is an advanced technique for replacing the default compiler assumptions with actual data collected by running a specially-instrumented executable. This requires compiling the program twice, first to create an instrumented executable and again to use the information collected by that executable.

Feedback-based optimization uses these options:

- `-ffeedback-generate`

  This option inserts extra instrumentation code that collects information for later feedback-directed optimization via the `-ffeedback-use` option. Use the `-ffeedback-generate` option when you compile individual source files. You must combine it with the `-ffeedback-emit` option at the link step to specify where the raw feedback should be written.
- `-ffeedback-generate-compiler`

This option is similar to `-ffeedback-generate`, but only instruments code to collect data for linker feedback-directed optimization via the `-ffeedback-use-linker` option. Use the `-ffeedback-generate-linker` option when you compile individual source files. You must combine the `-ffeedback-generate-linker` option with the `-ffeedback-emit` option at the link step to specify where the raw feedback should be written.

- `-ffeedback-generate-linker`

  This option operates the same way as `-ffeedback-generate`, but it only collects data for the link step, not for the compilation step.

- `-ffeedback-emit=`*`directory`*

  This option specifies a directory name into which the raw feedback data will be emitted. Each instrumented process creates a unique file in the specified directory. The `tile-convert-feedback` tool converts the contents of a raw feedback directory, which may contain data from many different processes, into a single feedback file suitable for use with the `-ffeedback-use` option.

- `-ffeedback-use=`*`file`*

  This option performs feedback-based optimization using the feedback data in the specified file, which was created by the `tile-convert-feedback` tool.

- `-ffeedback-use-compiler=`*`file`*

  This option performs compiler feedback-based optimization using the runtime profile data in the specified file produced by the `tile-convert-feedback` tool.

- `-ffeedback-use-linker=`*`file`*

  This option performs linker feedback-based optimization using the runtime profile data in the specified file produced by the `tile-convert-feedback` tool.

For more information on using these options, see the chapter on feedback optimization in the *Multicore Development Environment Optimization Guide* (UG515).

4.10 Using the Assembler (tile-as)

`tile-as` is a port of the GNU assembler `as` to the Tile Processor. It assists in the compilation of `.c` and `.cc` files. `tile-as` is invoked by the `tile-gcc` compiler driver if a `.S` file or a `.s` file is specified as input. `tile-as` supports the entire instruction set, including bundling operations into VLIW instructions.

For example:

```
some_label:
( move r11, r22; addi r22, r22, 32 }
( addi r8, r18, 1; addi r18, r18, 32; ld r3,r4 }
( addi r16, r16, 1; j some_label }
```

This example represents three instructions in a loop: the first and third instructions have two operations, and the second instruction has three operations.

A good way to learn `tile-as` syntax is to run `tile-gcc` with the `-S` flag and study the resulting `.s` file, which can be edited and passed to `tile-as`.

For more information, including how to write Tile assembly code, see "Tile Processor Assembler (`tile-as`)" in the Tilera MDE Document Index.

CHAPTER 5 DEBUGGING

This chapter provides information about debugging Tilera applications:

Note: Debugging works best with applications compiled with `-g -O0`.

5.1 Methods of Debugging

You can use several methods to debug an application:

- Using `tile-gdb` by means of `tile-monitor`.

  See Section 5.2 “Debugging Using tile-gdb and tile-monitor” on page 29.

- Using `gdb` for standard Linux-style debugging.

  See Section 5.3 “Standard Linux-Style Debugging By Means of Native gdb” on page 30.

- Using the Tilera IDE.

  The Tilera MDE also includes `tile-eclipse`, an Eclipse-based development/debugging IDE, which supports debugging of entire applications in a UI environment that manages the various processes and debuggers for you. For more information on using the IDE for debugging, see the chapter on “Debugging an Application in the IDE” in *Using the Tilera IDE* (UG511).

5.2 Debugging Using tile-gdb and tile-monitor

Debugging of processes on the Tile Processor is similar to normal debugging of Linux processes: You attach a debugger to each process for which you want to examine state or exercise debugging control. The only difference is that you run the debugger on an x86 host and communicate remotely with the Tile process through a port opened by `tile-monitor`.

The Tilera MDE includes `tile-gdb`, a debugger that runs on an x86 host and can be used to remotely debug a Linux process running on the Tile Processor. The `tile-gdb` utility is based on GNU GDB 6.8, and supports the standard GDB commands for stepping, examining program state, and so forth.

To use `tile-gdb`, you need to provide arguments to `tile-monitor` to indicate which process(es) should be debugged. These include:

- `--debug -+- exe` *`args`* `-+-` — launches an executable on the Tile Processor, and debugs the executable’s process

- `--debug-tile x,y` — debugs any process that runs on the specified tile
- `--debug-on-crash` — debugs any process that crashes

  With `--debug-on-crash`, if the application crashes, the shepherd will emit a message that you can cut and paste into a host machine to attach `gdb` and debug the application. The `gdb` command `bt`, as a useful example, does a stack trace.

For more information on `tile-monitor` arguments relating to debugging, see the `tile-monitor` man page.

When `tile-monitor` debugs a process, it puts the process in a "frozen", debuggable state, then prints instructions for starting an instance of `tile-gdb` and the commands you need to use to attach it to the process. You can then use the normal GDB commands to set breakpoints, resume, and step the program, as well as to examine the contents of variables, and the like.

5.2.1 Tips on Debugging with tile-gdb and tile-monitor

- `tile-monitor` provides a mechanism to let you quickly see the state of your application. It can connect `gdb` to all applications it has launched or is running and do a stack trace. Use `Ctrl-\`.

  This works even during a run when you do not have a `Command:` prompt.
- Use the `tile-monitor` option `--debug-on-crash` when you run an application.

  If the application crashes, the `shepherd` will emit a message that you can cut and paste into a host machine to attach `gdb` and debug the application. The `gdb` command `bt`, as a useful example, does a stack trace.

5.3 Standard Linux-Style Debugging By Means of Native gdb

Another tool shipped in the `tile/usr` hierarchy is the `gdb` debugger.

Complete documentation for the `gdb` debugger can typically be found on the host Linux where the MDE is installed, for example by means of the `info gdb` command. The `gdb` command can be used to launch an application for the purpose of debugging, or to attach after the fact to an already-running application to inspect its state or prepare to debug a crash.

In addition, if core dumps are enabled on the Tile Linux system (for example by running `ulimit -c unlimited` prior to running the application), then `gdb` can be used to debug the core dump images as well.

Note that `/usr/bin/gdb` relies on the `ncurses` library, which is also not included by default on the `initramfs`. If you are uploading files one at a time, you will also need to specify the `/usr/lib/libncurses.so.5.6` shared object and the `/usr/lib/libncurses.so.5` symbolic link.

For more information, see http://www.gnu.org/software/gdb/documentation/.

5.4 Linux Debugging Tips

- The Linux boot argument `crashinfo` should dump some information in the console if a crash occurs.
- To turn on core dumps, use the Linux argument `TLR_COREDUMPS`, which is documented in the *Multicore Development Environment System Programmer's Guide* (UG509).

  This argument is processed by the `$TILERA_ROOT/tile/etc/rc` script and simply runs the following: `# ulimit -c unlimited`

5.5 Debugging Tips Using tile-btk

To print out program counters (PCs) on every tile:

```
$ tile-btk -f /path_to_my_config_file.py
>>> start_protocol_mode()
>>> debug.print_pcs()
```

Or, to also print out the return address of the caller routine:

```
>>> debug.print_pcs_and_lrs()
```

5.6 Debugging Tips Using tile-addr2line

You can use `tile-addr2line` to look up the PCs and get the source code and list all the PCs that you want:

- `-f` for function names
- `-e` for directories and executable

For example, if hypervisor code was linked with addresses starting with `0xfexxxxxx`, run this command:

```
$ tile-addr2line -e $TILERA_ROOT/tile/boot/hv -f <0xfe rest of PC> \
   <0xfe rest of another PC>
```

Similarly, for Linux addresses starting with `0xfdxxxxxx`:

```
$ tile-addr2line -e $TILERA_ROOT/tile/boot/vmlinux -f 0xfd2758a0
```

Note: If the PC is inside a commonly called utility (and because the whole stack is not printed out), you can sometimes get better information by modifying the code to in-line the routine (in a `.h` file) and rebuilding the kernel.

CHAPTER 6 VIEWING TILE PROCESSOR ARCHITECTURE TRACES USING TILE-TRACE-VIEW

This chapter describes how to generate traces and use the `tile-trace-view` graphical viewer to view them:

6.1 Using tile-monitor for Tracing

Tracing produces detailed, cycle-by-cycle results for the simulator platform. Tracing produces files that contain information down to the machine-cycle level. You can view these files either as text, or by using `tile-trace-view`, which provides a graphical user interface (GUI) to display trace data.

The simulator tracing options all start with `--trace-`.

To focus tracing on specific program segments, use the Simulator Control API, which enables controlling tracing options under program control.

To turn tracing on and off around a section of code, use:

```
#include <sys/simulator.h>
sim_set_tracing(SIM_TRACE_CYCLES | SIM_TRACE_DISASM);
        :
// your code
        :
sim_set_tracing(SIM_TRACE_NONE);
```

6.2 Where to Find Documentation Information

See the following:

- For information about the Simulator Control API, see the *Multicore Development Environment Optimization Guide* (UG515).
- For information about using `tile-monitor` with the tracing options, see the "Tracing Options" section in the "Tile Processor Simulator (tile-sim)" in the MDE Document Index:

  `$TILERA_ROOT/doc/html/tile-sim.html#trace-options`
- For information about `tile-monitor`, see Chapter 2: Using tile-monitor.

6.3 What Is tile-trace-view?

`tile-trace-view` is a standalone graphical viewer for trace files generated by the `tile-monitor --trace-tile-view` simulator option. These traces provide information on what each tile is doing, such as executing, or stalled on some resource.

To generate trace files, use a command like this:

```
$ tile-monitor --sim-args -+- --trace-tile-view prefix -+-
```

Trace file names are prepended with the specified prefix.

The `--trace-tile-view` option is a shorthand for specifying several of the individual tracing options together with `--output-file`.

The tracing options all start with `--trace-`.

`tile-trace-view` is a Java program. In the Linux distribution, a launcher script named `tile-trace-view` is available in the `bin` directory.

6.4 Example of tile-trace-view Trace Output

Assume this trace:

```
Cycle 6250 (0,0) 0x00022240 __sflags+0x20   { movei $9, 0 ;
                                              movei $10, 0 }
Cycle 6251 (0,0) 0x00022248 __sflags+0x28   { addi $11, $3, 1 }
Cycle 6252 (0,0) 0x00022250 __sflags+0x30   { lb $11, $11 }
Cycle 6253 (0,0) 0x00022258 __sflags+0x38   { seqi $12, $11, 43 ;
                                              bz $11, __sflags+0x50 }
Cycle 6253 Stalled on mem_l1d_hit_on_read of 0x413f1
Cycle 6255 Stalled on branch_mispredicted
Cycle 6257 (0,0) 0x00022270 __sflags+0x50   { or $13, $10, $9 }
Cycle 6258 (0,0) 0x00022278 __sflags+0x58   { sw $4, $13 }
```

Each line indicates activity on one or more cycles. In the first line, for cycle 6250, the tile at location (0,0) is executing at program counter location 0x00022240, which is in the routine `sflags`, at 0x20 bytes beyond the start. The instruction being executed has two `movei` (move immediate) operations.

Farther down we see a stall, where initially the instruction is stalled reading from the L1 data cache, and then on mispredicting the branch direction.

6.5 Output Files Produced

The output files that are produced when you use `--trace-tile-view` option with `tile-monitor` are named like this:

- `prefix.xml` — aggregated statistics file
- `prefix.tile*.router` — network trace files
- `prefix.tile*.mainproc` — Tile Processor trace files

You can also generate interval profile files with `tile-monitor` by using the `--profile-interval` option along with the `--xml-stats` or `--xml-profile` options. This generates a separate XML profile file per interval, which enables you to trace much longer runs at a coarser granularity level.

6.6 Passing Arguments

When you run the `tile-trace-view` command, you can pass as arguments the list of trace files you want the program to process and summarize. For example:

```
$ tile-monitor [...] --sim-args -+- --trace-tile-view ttv -+- [...]
$ tile-trace-view ttv.*
```

You can also select the list of trace files from within the trace viewer.

To select the list of trace files from within the trace viewer:

1. Run `tile-trace-view` without arguments.
2. Click File in the menu bar, then Open.
3. Use the file browser to navigate to the directory containing the trace files.
4. Multi-select (click the first and Shift-click the last) the set of trace files (`*.xml`, `*.router`, and `*.mainproc`).
5. Click OK.

6.7 Displaying and Analyzing the Views

The `tile-trace-view` command provides three views, or visualizations:

- Trace Chart
- Scatter Plot
- Network Animation

A mouse right-click displays a context menu, including Zoom choices and a Select View dialog, which you can use to navigate between views. The dialog is initially populated with a cycle and trace derived from the right-click location. These can be edited, and used to select a different view focused on the chosen cycle and trace.

6.7.1 Trace Chart

The graph's horizontal axis is time, in cycles; the vertical axis is the tile number. For each tile, events are displayed in horizontal "bands," a broad one on top for the processor, and a narrower one on bottom for the selected network, as shown in Figure 6-1.

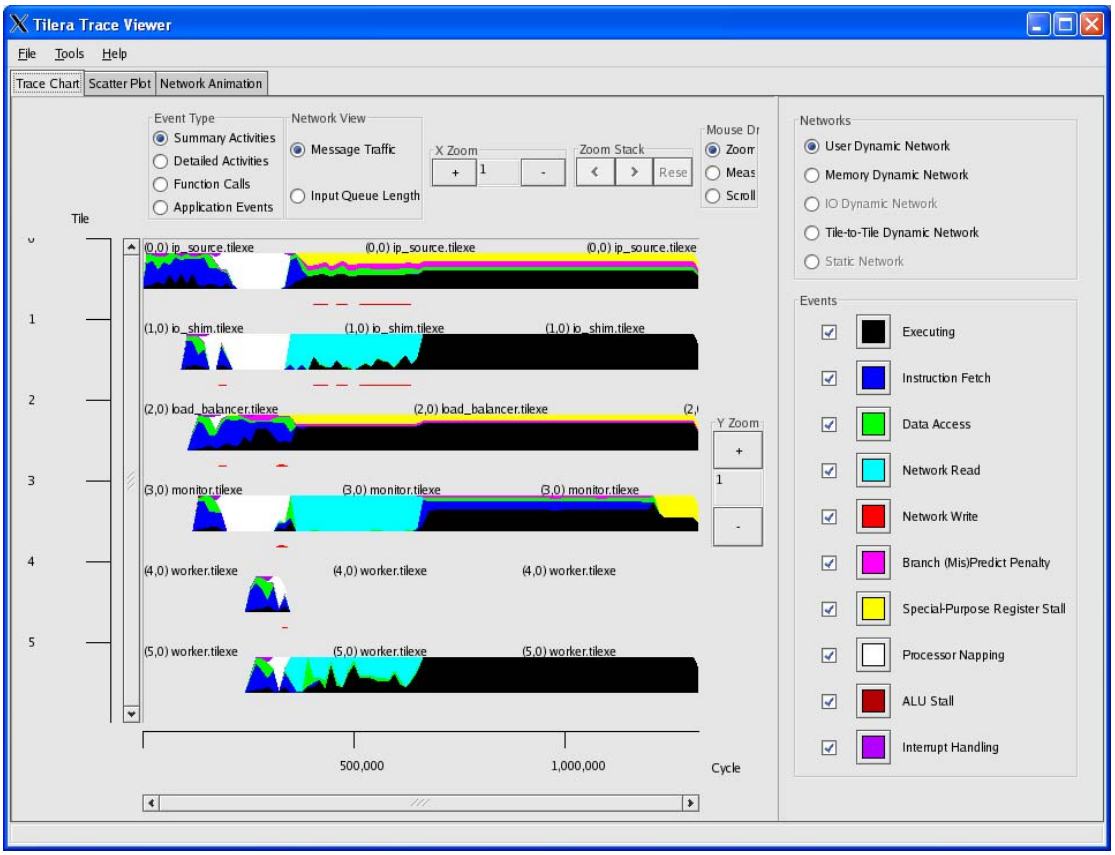

Figure 6-1: Trace Chart Screen

Each band is a stacked bar chart, with colors corresponding to time spent in various “events” such as processor execution, stalls of various types, and network traffic destinations. The events are color-coded according to the legend on the right.

Events displayed in the legend are selectable: you can turn on and off individual events (for example, to de-clutter the display or pick out the contribution of specific events). You can change the color of an event type by clicking its button in the legend.

To get more information about an event, hover the cursor over it. To make it easier to select a specific event, zoom in.

The two ways to zoom in are to use the X and Y Zoom controls, or to “rubber band.” To do the latter, select Zoom In in the Mouse Drag box, left-click in the graph, and then while holding the left mouse button down, drag the corner of the bounding box. Release to zoom. The Zoom Stack acts like browser backwards and forwards buttons, so one can revert to a previous zoom state, and then return back and so on.

When zoomed in, you can scroll in the X and Y dimensions, by using the scroll bars or by selecting the Scroll option in the Mouse Drag box. The latter acts as a “grab scroll,” moving the chart view along with the mouse. Grab scroll is also available via control-left-click regardless of the Mouse Drag selection.

If you accidentally zoom in too far, that is by clicking on the graph and selecting a very small region, just click the left arrow in the Zoom Stack controls to return to the previous zoom size.

Views can only display data that is available, depending on the trace or profile files loaded and the data available in them.

A number of different views are provided:

- Summary Activities — In the default view, detailed events are summarized into higher-level categories.
- Detailed Activities — Detailed events are broken out individually.
- Function Calls — Time slices are colored according to their location in the program, rather than the event type.
- Application Events — Application-level events are charted, as derived from BEGIN/END macros placed in the application source. (In this view, the Network View options below are not available.)
- Message Traffic — Show network traffic events, namely words being sent from a network port.
- Input Queue Lengths — Show the length of the network input queues, with a separate histogram per source (the tile processor and the four cardinal directions North, East, West, and South).

You can display interval profiles in the Trace Chart by loading the `prefix.xml` files.

6.7.2 Scatter Plot

As its name suggests, this visualization shows events distributed over time (cycles on the horizontal or X axis) and location (instruction or data addresses on the vertical or Y axis). One main processor trace at a time is selected in the right-hand-side legend, as shown in Figure 6-2.

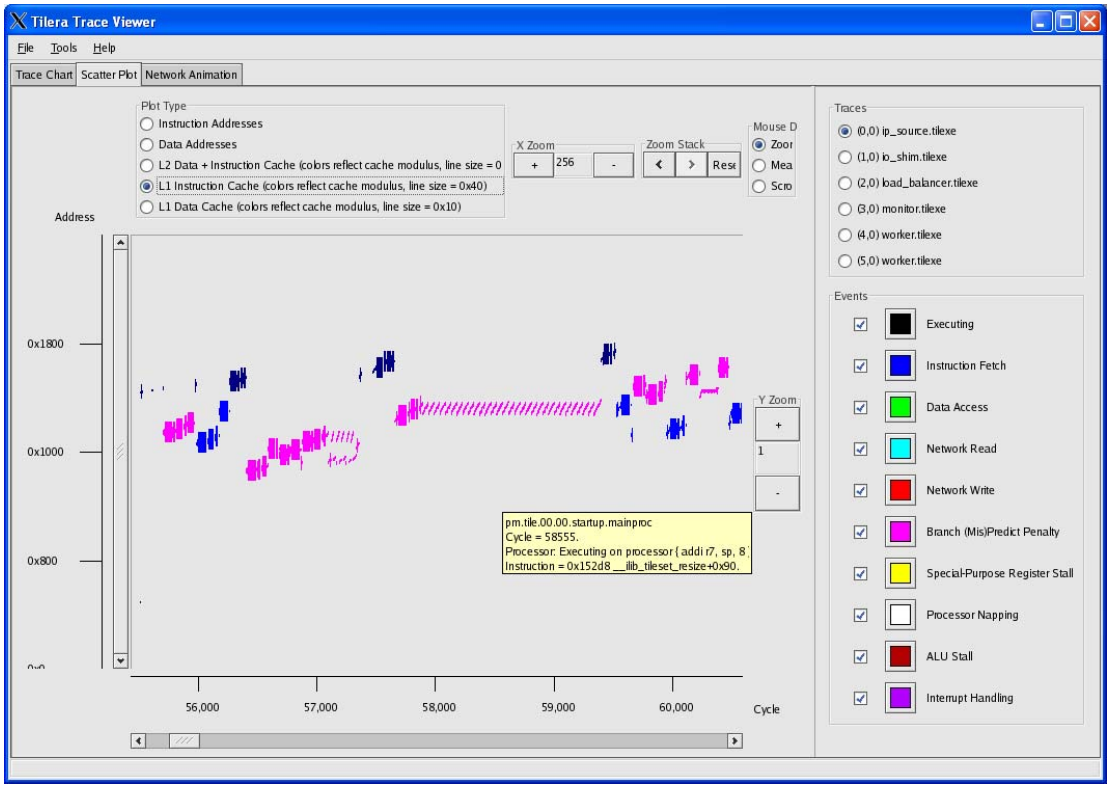

Figure 6-2: Scatter Plot Screen

Five variants are provided: all instruction or data addresses, and three special modes focused on cache effects in the level 2 unified cache or level 1 instruction or data caches.

In the cache variants, the coloring is done according to the virtual address modulo the cache associativity set size rather than the event type. This makes it easier to pick out potential cache conflicts, which typically look like alternating cache misses at close-by vertical positions with different colors, which implies objects at different addresses mapping to the same cache lines.

Zooming, scrolling, and so on work as they do in the Trace Chart.

6.7.3 Network Animation

This visualization, shown in Figure 6-3, shows the state of the entire chip over time.

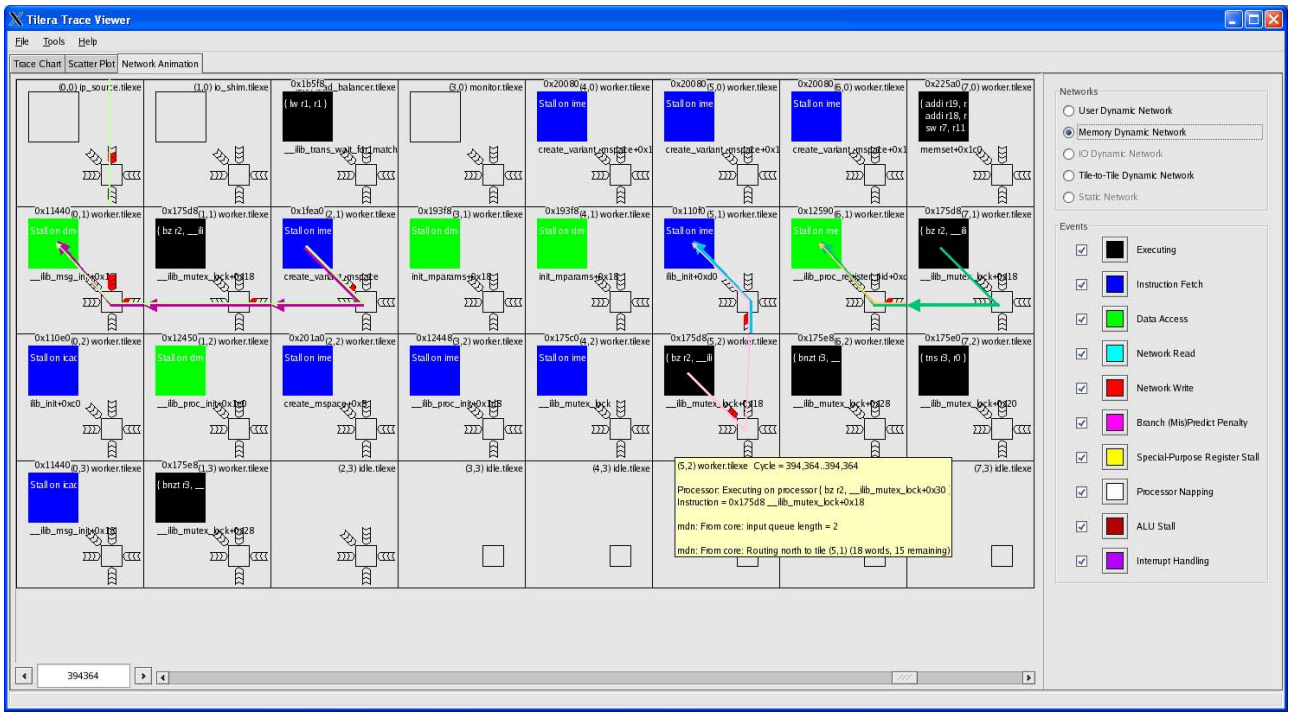

Figure 6-3: Network Animation Screen

In each tile, the central square represents the processor, and is colored according to event (executing instructions, stalls, and so on). The lower-right-hand area represents the router for the selected network, with five input queues for each of the cardinal directions and the processor itself. Network packets are indicated by arrows colored according to the destination tile. Hovering the mouse over a tile gives more detailed information on what's going on in the current cycle.

To scroll, use the controls on the bottom edge. On the left, use the buttons to scroll backwards and forwards a single cycle at a time. On the right, use the scrollbar for more global motion. (Tip: the scrollbar buttons will auto-repeat, giving a quick-and-dirty movie.)

CHAPTER 7 COLLECTING AND ANALYZING PERFORMANCE INFORMATION

This chapter describes how to collect performance information and analyze it:

7.1 Overview of Profiling on the Tile Processor

Profiling on the Tile Processor is similar to profiling in any Linux environment: you run your program and capture cumulative statistics about its overall behavior, then examine these after the run using a presentation tool.

The Tilera MDE includes support for low-impact, on-the-fly profiling: you do not need to instrument your code to take advantage of profiling support.

When running programs on the Tilera Simulator, you can simply ask the simulator to generate profiling statistics as the program runs. These are very detailed, because the simulator can basically "see" everything that it simulates:

```
$ tile-monitor --image 4x4 \
    --sim-args -+- --xml-profile profile.xml -+- \
    --upload myapp myapp \
    --run -+- myapp -+- \
    --quit
```

For profiling on hardware, the Tilera MDE includes the OProfile profiling utility, and `tile-monitor` provides a number of arguments or commands that allow you to easily request profiling for an application:

```
$ tile-monitor \
    --upload myapp myapp \
    --profile-init \
    --profile-start \
    --run -+- myapp -+- \
    --profile-stop \
    --profile-capture profile_dir \
    --profile-analyze profile_dir \
    --quit
```

The `--profile-init` option performs any initialization (uploading files, initializing flags state, etc.) required by the currently selected profiling tool. The `--profile-start` option begins profile data collection. The `--profile-stop` option halts profile data collection, and implicitly invokes the `--profile-dump` option to perform any necessary flushing of profiling state to ensure that the data is be available to the `--profile-capture` option, which downloads the current profiling data into the specified host-side directory, so that it can then be analyzed by the `--profile-analyze` option.

For either simulator or hardware profiling, the above commands generate a `profile.xml` data file that you can load into the Tilera IDE's Profile View for display. The Profile View allows you to filter, sort, and export profiling statistics, so that you can drill down to the area of interest in your application.

For a more detailed treatment of profiling in general, see the file `doc/html/profiling.html` in your MDE installation directory.

7.2 Methods of Profiling

Tilera supports these methods of profiling at the command line:

- Hardware platform: OProfile. This method uses `tile-monitor` commands such as:
  - `profile-init`
  - `profile-start`
  - `profile-capture` *`directory`*
  - `profile-analyze`

  You can use OProfile to create a `profile.xml` file that you can load into the Profile View in the IDE for viewing. The IDE provides interactive tools and call trace displays.

  Or you can use the `tile-opreport` command to view the data.

  See Section 7.3 "Hardware Profiling" on page 40.
- Simulator platform: the `tile-monitor --xml-profile` or `tile-monitor --xml-stats` options. These options create a `profile.xml` file that you can load into the Profile View for viewing.

  See Section 7.4.1 "Simulator Profiling" on page 49.

7.3 Hardware Profiling

Tilera® uses a customized version of OProfile to collect performance information for processes and threads running on the Tile Processor.

OProfile data is statistical, that is, rather than record every stack frame of your program, the profiler allows you to specify one or more events and count limits for each of them. Every time the count of a given event (a TLB miss, processor stall, and the like) reaches the specified limit, OProfile records the function in which the program was, plus some number of parent stack frames (the default is 20).

OProfile records this "raw" sampling data on disk in a samples directory. The Tilera IDE processes this sample data to generate a single `profile.xml` data file, which can be loaded into the IDE.

Profiling on hardware is recommended when you want to (a) profile your application in "real world" conditions and (b) do profiling in "real time", testing one or two conditions like stalls or cache misses at a time to isolate a problem.

OProfile captures performance information from applications and from the operating system. Any application can be profiled as is; no special rebuild or libraries are required. Profiling can be started and stopped at any time, allowing profiling of long-running applications without having to stop and restart them.

OProfile can report time spent in each single application, in each invocation of an application (process), or in any given thread or set of threads of a particular process. OProfile can show flat profiles in the style of `prof` or call graph profiles in the style of `gprof`. It can also be used to produce profile data readable by Tilera's implementation of `gprof`, `tile-gprof`.

OProfile can also produce source code annotated with profiling information, which allows you to find out which individual source lines need attention. It can produce annotated assembly listings to provide the most low-level and fine-grained information possible.

7.3.1 Getting Help on OProfile

See these sources:

- You can use the `tile-opreport` command for selecting, filtering, and displaying data.
- You can use the `tile-ophelp` command to display hardware counters.
- For a full explanation of OProfile in the Tilera environment, see "Profiling Applications from the Command Line with OProfile" in the MDE Document Index:

  `$TILERA_ROOT/doc/html/profiling.html`

  This document has an extended example and is the primary documentation for OProfile in the Tilera environment.
- For lower-level details on how to use OProfile, see the chapter on "Performance Analysis" in the *Multicore Development Environment Optimization Guide* (UG515).
- For information about the most important performance counters on the TILE-Gx processor family, see the appendix "TILE-Gx Performance Counters" in the *Multicore Development Environment Optimization Guide* (UG515).
- For detailed information on OProfile, see the OProfile documentation website at:

  `http://oprofile.sourceforge.net/docs/`

7.3.2 Getting a Listing of Hardware Events

For a list of hardware performance counter events, run `tile-ophelp`. The appropriate sampling interval differs from event to event. An event that ticks slowly needs a shorter interval, and one that ticks close to every cycle needs a longer interval.

For details about the performance counters, see the appendix "TILE-Gx Performance Counter Events" in the *Multicore Development Environment Optimization Guide* (UG515).

7.3.3 How to Use OProfile with tile-monitor

`tile-monitor` supports built-in commands that control configuring, starting, and stopping OProfile, as well as collecting and analyzing the results.

To profile a program using `tile-monitor` and OProfile, follow these basic steps:

1. Run `tile-monitor` with the appropriate `--pci`, `--upload`, and `--cd` arguments. At the `tile-monitor` prompt, enter these commands to initialize profiling:

   ```
   Command: profile-init
   Command: profile-kernel
   ```

   Omit the second command if you do not want kernel information. It does not add much overhead, but the information it collects is of interest only if you want to know where the operating system is spending its time.

   The `profile-init` and `profile-kernel` commands merely upload some files to the hardware's virtual file system, and only need to be invoked once for each profiling session.

2. Start the profiling session:

   ```
   Command: profile-start
   ```

3. Run the programs you want to profile by using the `tile-monitor run` or `launch` arguments. The `run` argument waits for the process to exit; the `launch` argument does not. You can also use the `-- executable arguments` format (which is similar to `run`).

4. Stop the profiler:

   ```
   Command: profile-stop
   ```

5. Capture the profile information:

   ```
   Command: profile-capture directory
   ```

   where *`directory`* is the *profiling data collection directory*.

6. Quit out of `tile-monitor`:

   ```
   Command: quit
   ```

You can do all the above steps in one command line:

```
$ tile-monitor \
    --upload your_program /remote/directory/your_program \
    --cd /remote/directory --profile-init --profile-kernel \
    [...]
    --profile-start --run - your_program arguments... - \
    --profile-stop --profile-capture directory --quit
```

7.3.4 Specifying Hardware Events to Record

You can use the `tile-monitor profile-events` command to specify hardware events that you want recorded:

```
Command: profile-events '--event=EVENT_NAME:nnnn --event=EVENT_NAME:nnnn'
```

You specify four events.

nnnn is the sampling interval (how often the event must occur before a sample is taken for that event).

Do not make the sampling interval too small (that is, less than 10,000 to 100,000, depending on the counter) or you might hang or crash Linux. (As a safety feature, if an excessive number of profile interrupts are detected, profile collection will be disabled, with a warning printed to the hardware console stream.)

By default, this command is assumed:

```
Command: profile-events '--event=ONE:110000 --event=VALID_WB:110000
--event=STALL:110000 --event=LOAD_STALL:110000'
```

This default setting specifies three special performance counter events:

- `ONE` is an event that occurs once every cycle, hence it is the processor cycle counter.
- `VALID_WB` is an event that occurs for every valid instruction bundle.
- `STALL` is an event that occurs for every instruction stall.
- `LOAD_STALL` is an event that occurs for every data stall.

`750000` is the profiling interval, or in this case, every 750,000 cycles, or 1,000 times a second on a 750 MHz processor.

Tilera's hardware events do not make use of `opcontrol`'s `unitmask` argument. Therefore, when using `opcontrol`, you can omit the argument or use `0`. For example, either of these two `opcontrol` commands is correct:

```
opcontrol --event=ONE:10000000
opcontrol --event=ONE:10000000:0
```

7.3.5 Enabling Call Graph Profiling

You can use the `tile-monitor profile-flags` command to overwrite existing OProfile flags.

By default, call graph profiling with a depth of 20 is set (that is, the `'--callgraph=20'` flag is set).

To set a depth of 5, for example, use this command:

```
Command: profile-flags '--callgraph=5'
```

The `profile-flags` command takes effect only after profiling is started or restarted. Thus, if profiling is already running, you will need to use the `profile-shutdown` and `profile-start` commands in order for the new flag to take effect.

7.3.6 Viewing OProfile Results with tile-opreport

You can also view OProfile results with `tile-opreport`.

`tile-opreport` is essentially a port of the OProfile command `opreport`, and thus provides similar options for selecting, filtering, and displaying data.

This section shows how to use `tile-opreport` to view profiling data generated by the network security program, `snort`, on the Tile Processor™ architecture.

The example uses `/tmp/profile` as the archive directory.

Follow these steps:

1. Run `tile-opreport archive:/tmp/profile`

```
CPU: CPU with timer interrupt, speed 750 MHz (estimated)
   Processes with a thread ID of 0
   Processes with a thread ID of 171
   Processes with a thread ID of 376
   Processes with a thread ID of 395
   Processes with a thread ID of 454
   Processes with a thread ID of 457
   Processes with a thread ID of 458
   Processes with a thread ID of 459
   Processes with a thread ID of 460
```

   This information is followed by a very wide report, with one column per thread run during profiling.

2. Run `tile-opreport archive:/home/joeuser/tmp/tilera /snort`:

```
Processes with a thread ID of 459
Processes with a thread ID of 460

      tid:459|          tid:460|
      samples|      %|  samples|      %|
      ------------------------------------
        31662   100.000     15880 100.000 snort
      tid:459|          tid:460|
      samples|      %|  samples|      %|
      ------------------------------------
          31628 99.8926     15842 99.7607 snort
             34  0.1074        38  0.2393 vmlinux
```

   Notice that we provided the full path name for snort on the command line. This report shows two threads for `snort`. One of the threads shows some Linux time.

3. Run `tile-opreport archive:/home/joeuser/tmp/tilera /snort -l -p /home/joeuser/tmp/tilera/tilera/apps`.

The `-l` flag produces a report with function names as well as application names. The `-p` argument is followed by the pathname where `opreport` can find an un-stripped version of the applications. This means the symbol table is still intact.

```
Processes with a thread ID of 459
Processes with a thread ID of 460

 [Some warnings about modification times, which you should ignore]

 samples  %        samples  %        image name  symbol name
 20967    66.2213  7253     45.6738  snort       netio_get_packet
 3439     10.8616  3207     20.1952  snort       pcap_wait_for_packet
 1131      3.5721  243       1.5302  snort       fa_search
 1105      3.4900  520       3.2746  snort       fpEvalHeaderSW_noncontent
 197       0.6222  104       0.6549  snort       pcap_live_loop
 176       0.5559  130       0.8186  snort       Preprocess
 167       0.5274  88        0.5542  snort       sfxhash_find_node_row
 156       0.4927  66        0.4156  snort       memcmp
 155       0.4895  71        0.4471  snort       IsPreprocBitSet
 139       0.4390  75        0.4723  snort       CheckTcpFlags
 122       0.3853  116       0.7305  snort       memset
 120       0.3790  123       0.7746  snort       ReassembleStream4
 100       0.3158  29        0.1826  snort       CheckAddrPort
 93        0.2937  102       0.6423  snort       fpEvalHeaderSW
 92        0.2906  82        0.5164  snort       fpFinalSelectEvent
 90        0.2843  111       0.6990  snort       Decode
IP
 81        0.2558  88        0.5542  snort       DecodeTCPOptions.cold
 81        0.2558  26        0.1637  snort       sfghash_find_node
 77        0.2432  48        0.3023  snort       GetSessionFromHashTable
```

This display is shortened for brevity. Notice that there are two columns: one for each of the two threads. With more threads, more columns appear. The functions are sorted by the number of samples in the first column.

4. To look at either thread, use the instruction:

   ```
   $ tile-opreport archive:/home/joeuser/tmp/tilera tid:459 /snort -l -p ~/tmp/
   tilera/apps
   ```

   The results are omitted. Notice the `argument tid:459`. The information is similar to the previous example, except only samples for the specified thread are displayed. These are representative of the threads. You could write `tid:459,460`.

5. To obtain more detail for the locations of the samples, add `-dg` to the command line. The filenames and line numbers will be added to the output:

```
$ tile-opreport archive:/home/joeuser/tmp/tilera tid:459 /snort -l -dg -p ~/
tmp/tilera/apps

   0001b368 20967   66.2213  netio_receive.c:120       snort
netio_get_packet
     0001b368 369       1.7599  netio_receive.c:120
     0001b370 386       1.8410  netio_receive.c:120
     0001b378 373       1.7790  netio_receive.c:120
     0001b380 367       1.7504  netio_receive.c:120
     0001b388 359       1.7122  netio_receive.c:120
     0001b390 378       1.8028  netio_receive.c:120
     0001b398 396       1.8887  netio_receive.c:120
     0001b3a0 363       1.7313  netio_receive.c:120
     0001b3a8 380       1.8124  netio_receive.c:120
     0001b3b0 350       1.6693  netio_receive.c:120
     0001b3b8 362       1.7265  netio_receive.c:136
     0001b3c0 360       1.7170  netio_receive.c:136
     0001b3c8 13897    66.2803  netio_receive.c:140
     0001b3d0 371       1.7694  netio_receive.c:140
     0001b3d8 363       1.7313  netio_receive.c:136
     0001b3e8 6         0.0286  netio_receive.c:140
     0001b428 1         0.0048  netio_receive.c:145
     0001b438 6         0.0286  netio_receive.c:145
     0001b470 1         0.0048  netio_receive.c:157
     0001b4b8 1         0.0048  netio_receive.c:157
     0001b4c8 11        0.0525  netio_receive.c:159
     0001b528 1115      5.3179  netio_receive.c:169
     0001b530 360       1.7170  netio_receive.c:169
     0001b538 392       1.8696  netio_receive.c:169
   00028e98 3439 10.8616  pcap.c:1278 snort  pcap_wait_for_packet
     00028ee8 1         0.0291  pcap.c:1288
     00028f18 1146     33.3236  pcap.c:1288
     00028f20 400      11.6313  pcap.c:1288
     00028f28 387      11.2533  pcap.c:1288
     00028f30 388      11.2823  pcap.c:1288
     00028f38 342       9.9448  pcap.c:1293
     00028f40 393      11.4277  pcap.c:1293
     00028f48 380      11.0497  pcap.c:1293
     00028fb0 1         0.0291  pcap.c:1308
     00029040 1         0.0291  pcap.c:1321
```

6. There is more than one listing for `netio_receive.c:120`, because there is more than one PC corresponding to this line number. We can see this by using just `-d` (do not use `-dg`):

```
vma      samples  %        image name               symbol name
0001b368 20967    66.2213  snort                    netio_get_packet
  0001b368 369       1.7599
  0001b370 386       1.8410
  0001b378 373       1.7790
  0001b380 367       1.7504
  0001b388 359       1.7122
  0001b390 378       1.8028
  0001b398 396       1.8887
  0001b3a0 363       1.7313
  0001b3a8 380       1.8124
  0001b3b0 350       1.6693
  0001b3b8 362       1.7265
  0001b3c0 360       1.7170
  0001b3c8 13897    66.2803
  0001b3d0 371       1.7694
```

```
      0001b3d8 363       1.7313
      0001b3e8 6         0.0286
      0001b428 1         0.0048
      0001b438 6         0.0286
      0001b470 1         0.0048
      0001b4b8 1         0.0048
      0001b4c8 11        0.0525
      0001b528 1115      5.3179
      0001b530 360       1.7170
      0001b538 392       1.8696
    00028e98 3439     10.8616  snort                    pcap_wait_for_packet
      00028ee8 1         0.0291
      00028f18 1146     33.3236
      00028f20 400      11.6313
      00028f28 387      11.2533
      00028f30 388      11.2823
      00028f38 342       9.9448
      00028f40 393      11.4277
      00028f48 380      11.0497

      00028fb0 1         0.0291
      00029040 1         0.0291
```

7. You can obtain results in a different way:

```
$ tile-opannotate --source archive:/home/joeuser/tmp/tilera /snort -p ~/tmp/
tilera/apps tid:460
```

This creates an annotated version of the source showing the location of samples. The example below provides an extract from this, which shows 1066 samples, about 6.7% of the total samples for the thread at the given line:

```
/*
     * Command line: opannotate --source archive:/home/joeuser/tmp/tilera /
snort -p /home/joeuser/tmp/tilera/apps tid:460
     *
     * Interpretation of command line:
     * Output annotated source file with samples
     * Output all files
     *
     * CPU: CPU with timer interrupt, speed 750 MHz (estimated)
     * Profiling through timer interrupt
     */
     *
    /* Total samples for file : "/u/ken/p4/apps/lib/libnetio/netio_receive.c"
     *
     *   7288 45.8942
     */

      1066  6.7128 :  if (queue.__user_part->__packet_receive_read ==
                   :      queue.__system_part-
>__packet_receive_queue.__packet_write)
                   :    return NETIO_NOPKT;
```

7.3.7 Looking at Call Graph Profiles

You can also look at call graph profiles, which present application time hierarchically. This allows you to see not only which functions account for the time in an application, but also attribute the time to the various functions' call sites. We can also see how much time particular functions spend in the functions they call versus executing directly:

```
$ tile-opreport -c --merge=tgid -p /home/joeuser/tmp/tilera/apps
archive:/home/joeuser/tmp/tilera /snort
```

Here, `-c` specifies call-graph profiling. Use `--merge=tgid` to get the sum all the data from all the separate threads). If you omit `--merge=tgid` and specify a particular thread instead (with `tid:`*XXX* where *XXX* is the thread ID of a particular process or thread we want to profile) we get the information shown in the sample below.

Only the first few lines of the result are shown:

```
samples  %        image name               symbol name
    -------------------------------------------------------------------------
  3         0.2171  snort                    fpEvalHeaderIp
  282      20.4052  snort                    fpEvalHeaderUdp
  1097     79.3777  snort                    fpEvalHeaderTcp
  710       7.8263  snort                    fpEvalHeaderSW_noncontent
  710      51.3748  snort                    fpEvalHeaderSW_noncontent [self]
  211      15.2677  snort                    CheckTcpFlags
  102       7.3806  snort                    CheckBidirectional
  73        5.2822  snort                    CheckIpOptions
  58        4.1968  snort                    UpdateNQEvents
  46        3.3285  snort                    CheckDsizeEq
  40        2.8944  snort                    CheckFragBits
  35        2.5326  snort                    ByteTest
  28        2.0260  snort                    CheckTtlEq
  24        1.7366  snort                    IpSameCheck
  21        1.5195  snort                    IpProtoDetectorFunction
  17        1.2301  snort                    CheckTcpAckEq
  13        0.9407  snort                    OptListEnd
  4         0.2894  snort                    CheckDstIP
-------------------------------------------------------------------------
  698     100.000   snort                    acsmSearchCodedDFA
  369       4.0675  snort                    fa_search
  369      52.8653  snort                    fa_search [self]
  326      46.7049  snort                    otnx_match
  3         0.4298  snort                    case_compare
-------------------------------------------------------------------------
  1         0.3861  snort                    flowcache_newflow
  1         0.3861  snort                    GetNewSession
  1         0.3861  snort                    ParsePattern
  1         0.3861  snort                    opt_tree_node_calloc
  2         0.7722  snort                    ps_tracker_get
  5         1.9305  snort                    SnortHttpInspect
  15        5.7915  snort                    Preprocess
  40       15.4440  snort                    fpEvalHeaderSW
  193      74.5174  snort                    DecodeEthPkt
  259       2.8549  snort                    memset
  259     100.000   snort                    memset [self]
-------------------------------------------------------------------------
```

The first entry indicates that 710 samples were collected while executing directly in the snort function `fpEvalHeaderSW_noncontent`. Additionally, 672 samples were collected in other functions called either directly or indirectly from `fpEvalHeaderSW_noncontent`. (Sum the sample counts for the functions below the main entry to derive this number.)

The functions listed below the main entry are the functions called directly from `fpEvalHeaderSW_noncontent`. The associated sample counts are totals for these functions and all their direct and indirect callees while `fpEvalHeaderSW_noncontent` is active (on the stack). The percentages are relative to the total for `fpEvalHeaderSW_noncontent`.

The functions listed above the main entry for `fpEvalHeaderSW_noncontent` are the direct callers of it. The associated sample counts are for `fpEvalHeaderSW_noncontent` when called by each function.

In particular, around 79% of the time spent in `fpEvalHeaderSW_noncontent` and its direct and indirect callees happened when it was called form `fpEvalHeaderTcp`. Around 51% of the time when `fpEvalHeaderSW_noncontent` was active was spent in the function itself. About 15% of the time was in `CheckTcpFlags` and any functions called directly or indirectly by `CheckTcpFlags`.

Call graph profiling is the default for `tile-monitor` profiling. It is more intrusive than profiling without call graph information, but it provides much more information. You can disable the graph profiling function by using the following `tile-monitor` command (issued before `profile-start`):

```
Command: profile-extra ''
```

7.3.8 Programmatically Starting and Stopping Profiling

Profiling can be started and stopped by programs running on the Tile Processor. When you configure `tile-monitor` for profiling, include the following `tile-monitor` command:

```
Command: profile-kernel
```

Then a program can invoke the program `opcontrol-kernel` as follows:

```
$ opcontrol-kernel --start --separate=kernel,thread
```

where `opcontrol-kernel` is just a wrapper around `opcontrol` with some specific options. `opcontrol-kernel` is a shell script that is dynamically generated and uploaded.

To stop collecting profiling data, use:

```
$ opcontrol-kernel --stop
```

`opcontrol-kernel` can be used to perform all the more advanced functions of `opcontrol`; it merely provides the kernel profiling information as a convenience.

7.4 Profiling from the Command Line

7.4.1 Simulator Profiling

On the simulator, profiling is relatively simple: you just add the `tile-sim --xml-profile` argument, and specify an output filename, and run your program as usual. The simulator captures profiling data as it runs, and when the application exits it generates the specified XML file containing the profiling data.

(Note that `--xml-profile` is a simulator argument, so you need to use `tile-monitor`'s `--sim-args` argument to tell `tile-monitor` to pass it through to the simulator.)

For example, here is a `tile-monitor` command line that performs a profiling run on the simulator:

```
$ cd /home/scratch/ide/projects/hello_world_pthread
$ tile-monitor --simulator --image 4x4 \
       --sim-args -+- --xml-profile profile.xml -+- \
       --upload hello_world hello_world \
       --run -+- hello_world -+- \
       --quit
Main process: Hello world!
Thread on tile 0: Hello, Brave New World!
Thread on tile 1: Hello, Brave New World!
Thread on tile 2: Hello, Brave New World!
Thread on tile 3: Hello, Brave New World!
Started 4 pthreads
```

This creates an XML profile output file named `profile.xml` in the `hello_world_pthread` application directory.

7.4.2 Profiling on the Hardware, from the Command Line

For hardware profiling, `tile-monitor` provides a number of command-line options for configuring OProfile, starting/stopping profiling, and capturing/analyzing the resulting profile data.

In general, the pattern to follow in using these arguments is:

- `--profile-init` and optionally `--profile-kernel` to upload OProfile support to the simulator or hardware
- `--profile-start` to start profiling before your application runs
- `--run` to invoke your application, so that `tile-monitor` waits for the application to finish running
- `--profile-stop` to stop profiling
- `--profile-capture` to capture the raw OProfile data from the hardware/simulator's virtual file system to the local host
- `--profile-analyze` to process the raw OProfile data into a `profile.xml` file that can be viewed in the IDE

For example, here is a `tile-monitor` command line that performs a profiling run on hardware:

```
$ cd /home/scratch/ide/projects/hello_world_pthread
$ mkdir profile_samples
$ tile-monitor \
       --upload hello_world hello_world \
       --profile-init --profile-kernel --profile-start \
       --run -+- hello_world -+- \ --profile-stop \
       --profile-capture profile_samples/run1 \
       --profile-analyze profile_samples \
       --quit
Using 2.6+ OProfile kernel interface.
Reading module info.
Using log file /var/lib/oprofile/samples/oprofiled.log
Daemon started.
Profiler running.
Main process: Hello world! Thread on tile 0:
Hello, Brave New World!
Thread on tile 1: Hello, Brave New World!
Thread on tile 2: Hello, Brave New World!
Thread on tile 3: Hello, Brave New World!
Started 4 pthreads
Stopping profiling.
Processing sample directories...
```

```
Processing sample directory '/home/scratch/ide/projects/hello_world_pthread/
    profile_samples/run1'...
Processed all sample directories.
Writing profile output to '/home/scratch/ide/projects/hello_world_pthread/
    profile_samples/run1/profile.xml'...
Found 'ONE' event, with event interval: 700000
Set multiplier for 'Cycles (Estimated)' statistic to: 700000
Profile output completed.
```

This creates an XML profile output file named `profile.xml` in the subdirectory `profile_samples` of the `hello_world_pthread` application directory.

Note that the `run1` directory also includes the raw OProfile sample data captured from the application run. This directory can be used with OProfile command-line tools, such as `opreport`, and the like. For more information about OProfile, see the `profiling.html` file in the `docs/html` directory in the MDE installation. (This file is called "Profiling Applications from the Command Line with oProfile" in the MDE Document Index.)

Profiling an application adds significant overhead in terms of simulator performance. Because of this, it is often desirable to profile only a "section" of a target application. In order to do this, you can use `tile-sim`, which provides an argument for disabling profiling on program startup and allowing a programmatic enabling/disabling of the profiler.

`--profile-disable-at-startup` disables profiling for the simulator, but, when used in conjunction with the simulator argument `--xml-profile`, configures the simulator for profiling. This allows the user to enable/disable profiling programmatically using the `sim_profiler_enable() /sim_profiler_disable()` APIs. For more information about `sim_profiler_enable()` and `sim_profiler_disable()`, see the *Application Libraries Reference Manual* (UG527).

Your `tile-monitor` command would look something like this:

```
./bin/tile-monitor --image 4x4 --sim-args - --xml-profile my_app
--profile-disable-at-startup - -- ./my_app
```

And your source code would look something like this:

```
#include <arch/sim.h>

int
main()
{
    // uninteresting init code
    init();

    sim_profiler_enable();
    benchmark();
    sim_profiler_disable();

    // cleanup code
    cleanup();

   return 0;
}
```

For information on how to create a `profile.xml` file on the simulator when running from the command line, see Section 7.4.1 "Simulator Profiling" on page 49.

7.4.3 Programmatically Enabling/Disabling Profiling Support on the Simulator

To focus profiling on specific program segments, use the Tilera Multicore Components (TMC™) library, which enables controlling profiling options under program control.

To turn profiling on and off around a section of code, use:

```
#include <arch/sim.h>

sim_profiler_enable();

:

// your code

:

sim_profiler_disable();
```

Note: All eight `sim_profiler_*` functions work only on the simulator, not the hardware.

For more information, see the "Tilera Multicore Components (TMC) Library" chapter in the *Application Libraries Reference Manual* (UG527).

CHAPTER 8 TOOLS FOR REMOTE ACCESS AND NETWORKING

This chapter describes tools used for remote access and networking:

- Section 8.1 Remote Interactive Access
- Section 8.2 Remote File Access
- Section 8.3 Establishing a Network Connection

Note: Much of the information in this chapter involves `tile-monitor`. For more information, see the "Tile Processor Monitor" (`tile-monitor.html`) in the "Target Platform Tools" section of the MDE Document Index or the man page at `tile-man tile-monitor`. (These are identical.)

8.1 Remote Interactive Access

This section describes various techniques to get interactive access to the Tile board from another system.

8.1.1 Interactive Access with tile-monitor

The `tile-monitor` command provides a simple way to run commands remotely on the Tilera board. Once a channel has been established, you can simply enter the command `run ls` (for example) to run an `ls` command on the Tilera board. You can also launch a command that runs asynchronously by means of the launch command; this is equivalent to launching a command in the background from the shell.

The `tile-monitor` command can provide interactive access even in the absence of a network connection to the Tile Processor, using a PCI link from another Linux system (using the `--dev gxpci0` option) or can run using an existing network connection (using the `--net` option).

8.1.2 Interactive Access with telnet

`telnet` is enabled by default in the Tilera Linux configuration. If no `telnet` daemon is required, you can disable it with the `TLR_TELNETD=n` boot environment variable.

In the default configuration, the `root` account has no password, so when you `telnet` to the board, you must enter `root` as the user name; you will not be prompted for a password.

A network connection is required to connect by means of `telnet`; see Section 8.3 "Establishing a Network Connection" on page 56 for information.

8.1.3 Interactive Access with ssh

The `ssh` protocol allows for more sophisticated connections to the Tilera board. The user can specify various protocols for authentication and encryption, and a file transfer mode is included in the protocol as well.

A network connection is required to connect by means of `ssh`; see Section 8.3 "Establishing a Network Connection" on page 56 for information.

Configuring and Running the OpenSSH Daemon

By default, the `initramfs` image includes the Dropbear SSH (Secure Shell) server, configured with a fixed private key. Use standard SSH clients to connect to the server. The Dropbear SSH server includes client-side support for SSH and SCP (Secure Copy) connections outbound from the Tile board as well.

If you install the heavier-weight OpenSSH (OpenBSD Secure Shell) server, `sshd`, you must disable use of the Dropbear server. Do this by specifying `TLR_SSHD=n` on the boot command line. You must then upload or mount the appropriate binaries and configuration files prior to running the daemon. See Section 8.2 "Remote File Access" on page 54 for more information.

Note that in addition to some files in `/usr`, you need the `/etc/ssh` directory to be present on the Tile chip.

Once the `/usr/bin/ssh` or `/usr/bin/scp` binary is present on the Tile chip, you can use it to create `ssh` or `scp` connections (respectively) from the chip to other `sshd` servers, without having to upload any other executables or configuration files, or perform any other configuration.

Before starting the server for the first time, you must create private keys for the server. These keys reside in the `/etc/ssh` directory, so if the directory is on persistent media, you only need to run these commands once. The commands to run are:

```
ssh-keygen -t rsa1 -f /etc/ssh/ssh_host_key -N ""
ssh-keygen -t rsa -f /etc/ssh/ssh_host_rsa_key -N ""
ssh-keygen -t dsa -f /etc/ssh/ssh_host_dsa_key -N ""
```

If you create a fresh filesystem image at each reboot, as is the default with Tilera Linux, you need to run these commands at each reboot, and your `ssh` clients need to tolerate frequently changing `ssh` server keys. Typically this means deleting the `~/.ssh/known_hosts` file on your `ssh` client systems, or editing out lines that reference the previous public key on the Tilera `ssh` server.

To start the server, simply run `/usr/sbin/sshd`. By default, the `/etc/rc` script does not try to start `sshd`, so you need to start it manually (or modify the `/etc/rc` script).

Once `sshd` is running, you should be able to run the command `ssh root@CHIP_IP` and log in without needing to enter a password. Similarly, you should be able to `scp` files back and forth.

8.2 Remote File Access

There are three methods of uploading and downloading files and directories to and from the Tile chip when working at the command line:

- Using `tile-monitor`
- Using `scp`
- Using the `pci-pipe` application
- Using `nfs`

8.2.1 Using tile-monitor to Upload or Download Files

The `tile-monitor` command provides a family of upload and download commands to move files back and forth between the host and the Tile board. For example, you can upload a directory `demo` in your home directory to the Tile board by means of the `upload demo /demo` command.

8.2.2 Mounting Directories on Tile Using tile-monitor

`tile-monitor` can remotely mount directories on the host filesystem onto the target filesystem using special mount points managed by the FUSE (file system in user space) library.

The `--mount`, `--mount-same`, and `--mount-tile` commands perform the actual mounting operations.

The `--here` command is shorthand for a common pair of operations: mounting the current working directory as itself, and then entering that directory on the target. This allows many programs to act as if they were actually running on the host.

The `--root` command is shorthand for uploading the `/etc`, `/sbin`, and `/var` directories, and mounting the `/bin`, `/boot`, `/lib`, and `/usr` directories.

8.2.3 Using scp to Upload or Download Files

You can use the `scp` command to upload and download files. To run `scp` from the Tile side simply requires arranging for `scp` to be included in the Tile filesystem. See `man scp` on the host Linux for more information on the `scp` command.

8.2.4 Using pci-pipe to Upload or Download Files

A fast method to upload and download files is to use the `pci-pipe` application, which involves running `pci-pipe` on the Tile board and `tile-pci-pipe` on the x86 Linux host.

For example, on the Tile board:

```
$ pci-pipe -r tar -xf -
```

and on the x86 Linux host:

```
$ tile-pci-pipe -w tar -cf - FILE/DIRECTORY [...]
```

8.2.5 Mounting Directories on Tile Using NFS

The network file system (NFS) is a traditional solution for mounting remote directories on Linux systems, and it can also be used with Tilera Linux. You must upload the `tile/lib/modules` directory from the MDE to the Tile system first, so that the NFS kernel module and other supporting modules are available to Tile Linux. (Note that the MDE has both a `lib/modules` and a `tile/lib/modules` directory; you must upload the latter, not the former.)

Once the kernel modules are available, you can use normal NFS mount commands on the TILE side, for example `mount nfsserver:/home /home`.

If you want read/write access to the files by means of NFS, you usually need to configure appropriate local users in the TILE `/etc/passwd` file to match the server; see Section 1.9.3 “Configuring Additional Users or Passwords” on page 10 for details.

A network connection is required to create an NFS mount; see Section 8.3 “Establishing a Network Connection” on page 56 for information.

Note: Because of the default configuration of the Tile Linux kernel, you might need to add the following parameters to the NFS mount arguments: `-o nfsvers=3,nolock`

8.2.6 Preloading Files into the Initramfs

While not strictly a form of remote file access, it is worth noting that you can also provide additional files (effectively “uploading” them) during initial boot by means of `tile-monitor`. Do this by placing all the additional files into a compressed `cpio` archive file (for example, `myfiles.cpio.xz`) and then specifying `--initramfs myfiles.cpio.xz` to `tile-monitor` when booting the chip. The compressed `cpio` archive is then a part of the initial boot sequence. Linux uncompresses the files in that archive after uncompressing its own built-in files from the `contents.txt` it was built with.

For information about how to do this, see the section on "Controlling the Linux Initramfs Boot Process" in the "Linux Interfaces and Customization" chapter of the *Multicore Development Environment System Programmer's Guide* (UG509).

8.3 Establishing a Network Connection

This section discusses how to set up such a network connection to run `telnet` or `ssh`, NFS, or `tile-monitor --net`.

8.3.1 Using Native Ethernet Devices

You must boot the kernel must with a suitable `TLR_NETWORK` boot environment variable, or run the `ifconfig` command to configure networking manually. For more information, see the *Multicore Development Environment System Programmer's Guide* (UG509).

Some Tilera Linux systems also include other Ethernet hardware; if they are configured for use by the Linux networking stack, they can also be used for remote access.

8.3.2 Using PCI for a Network Connection

In the absence of hardware Ethernet support into the Linux kernel, it is possible to set up networking over a PCI connection to a Tilera board.

Using tile-monitor to Tunnel Specific TCP Ports

The `tile-monitor` application has a `tunnel` command that allows the user to arrange for traffic arriving at host port "A" to be redirected to Tile port "B". For example, the `tunnel 2023 telnet` command would arrange for `tile-monitor` to open port 2023 on the local Linux host, then forward any received traffic into port 23 (the `telnet` protocol port) on the Tile board. Once such a connection is set up from the host side, any return traffic is then delivered back to the host side as well.

In this example, once the command has been run, you can run `telnet localhost 2023` on the same host as the `tile-monitor` process and receive the Tile board's login prompt, as if you were logging in across the network. Because `tile-monitor` is providing the tunneling service, it must stay up and running during the entire `telnet` session.

This approach does not work for having the Tile board initiate outbound connections; it only allows previously configured TCP tunnels inbound to the Tile.

Note: You can use `tile-monitor`'s `tunnel` option for `ssh`. For example, you can specify `tunnel 2022 ssh`. You must then tell the `ssh` client to use port 2022, typically with the `-p 2022` option. However, in this case the `ssh` client generates errors if it already has a public key for the host server, because it does not recognize that the tunnel is to the Tile board. To avoid this error condition, include a `StrictHostKeyChecking no` directive in the `ssh` config file (for example: `~/.ssh/config`).

8.3.3 Using tile-pci-net to Establish a Full Network Link

The MDE includes the `pci-net` and `tile-pci-net` applications to establish network connectivity between a TILE-Gx card and the host over PCIe. The basis for this solution is to create an IP tunnel over a PCIe character stream channel to deliver IP traffic between the TILEncore-Gx™ card and the host. For more information on the `pci-net` application, see the *Multicore Development Environment System Programmer's Guide* (UG509).

For the simple case, run `pci-net` on the TILEncore-Gx card, and `tile-pci-net` (as root) on the x86 host Linux. Run both programs in the background and leave them running.

Once both programs are running, the host Linux allows you to make connections to the Tilera board at address 192.168.100.201 (the IP of the PCI network link on the Tilera board), and the Tilera Linux board allows connections to be made to the host at address 192.168.100.101 (the IP of the PCI network link on the host Linux). By default, these connections are not configured as a default gateway on the TILE side; you can only reach the x86 Linux host machine. Routing for such connections could, in principle, allow wider network connectivity, although this is not currently supported by Tilera.

INDEX